TECHPULSE: A REAL-TIME HYBRID RETRIEVAL-AUGMENTED GENERATION SYSTEM FOR EMERGING TECHNOLOGY INTELLIGENCE

Progress Report

by

Aye Khin Khin Hpone (Yolanda Lim) st125970

Dechathon Niamsa-Ard st126235

AT82.9002 Selected Topic: Data Engineering and MLOps

Instructor: Dr. Chantri Polprasert

Teaching Assistants: Swaraj Bhanja, Suryansh Srivastava

Asian Institute of Technology

School of Engineering and Technology

Thailand

March 2026

ACKNOWLEDGMENTS

The authors express sincere gratitude to Dr. Chantri Polprasert, the course instructor, for providing guidance on the system design and evaluation methodology throughout this project. We also thank the teaching assistants, Mr. Swaraj Bhanja and Mr. Suryansh Srivastava, for their constructive feedback during development milestones.

Team Contributions. Aye Khin Khin Hpone led data ingestion, preprocessing, embedding, and the React frontend. Dechathon Niamsa-Ard led the RAG orchestrator, LLM fallback chain, AWS infrastructure (SAM templates), CI/CD pipeline, and evaluation framework. Both members contributed to the system architecture design, unit testing, and report writing.

ABSTRACT

Large language models (LLMs) offer advanced generative capabilities but are constrained by static training corpora, producing hallucinated or outdated outputs when queried about recent developments. Retrieval-Augmented Generation (RAG) mitigates this by conditioning generation on externally retrieved evidence; however, baseline vector-only retrieval often suffers from contextual imprecision.

This report presents TechPulse, a cloud-native Hybrid RAG system that synthesizes emerging technology intelligence under latency and cost constraints. The system ingests data from five heterogeneous sources—ArXiv, Hacker News, DEV.to, GitHub Trending, and curated RSS/Atom feeds—with deterministic preprocessing, token-based chunking, and semantic embedding via the MiniLM sentence-transformer. Embeddings are indexed in PostgreSQL with the pgvector extension for integrated relational filtering and vector similarity search.

Retrieval operates as a three-stage pipeline: metadata-constrained filtering, cosine similarity search, and lightweight reranking combining semantic similarity, lexical overlap, and exponential recency decay. An automatic LLM fallback chain (Groq $\rightarrow$ Bedrock $\rightarrow$ Ollama $\rightarrow$ HuggingFace) ensures generation availability. The system integrates dual-criteria drift detection, three-layer hallucination verification, and health monitoring. An evaluation framework comparing baseline and hybrid retrieval has been implemented using RAGAS metrics, grid search over reranking weights, and statistical testing (Wilcoxon signed-rank, Cohen's d, bootstrap 95% CI). This report documents the system architecture, pipeline design, implementation decisions, deviations from the original proposal, and remaining work.

Keywords: Retrieval-Augmented Generation; Hybrid Information Retrieval; Vector Databases; Cloud-Native Architecture; MLOps; Semantic Embedding; Emerging Technology Intelligence.

CONTENTS

ACKNOWLEDGMENTS **i**

ABSTRACT **ii**

1 Introduction **1**

1.1 Problem Description and Motivation 1
1.2 Research Questions 2
1.3 Objectives of the Study 2
1.4 Significance of the Study 3
1.5 Alignment Between Research Questions and Evaluation 3

2 Background and Related Work **5**

2.1 Retrieval-Augmented Generation 5
2.2 Embedding Models and Vector Search 5
2.3 Hybrid Retrieval and Reranking 5
2.4 RAG Evaluation Methodologies 6
2.5 Data and Concept Drift in Retrieval Systems 6
2.6 Hallucination Detection and Faithfulness Verification 6
2.7 MLOps and Production Machine Learning Systems 7
2.8 Research Gap 7

3 Methodology **10**

3.1 Overview of Technical Approach 10
3.2 Why This Approach is Suitable 10
3.3 Technical Contributions and Design Decisions 11
3.4 Baseline Retrieval Configuration 11
3.5 Implemented Hybrid Retrieval Configuration 12
3.6 Hallucination Verification Strategy 14
3.7 Optimization Objective 14
3.8 Tool Selection and Implementation Stack 14
3.9 System Context Overview 15

4 System Architecture and Pipeline Design **17**

4.1 Architectural Overview 17
4.2 Alignment with the AWS Well-Architected Framework 17
4.3 Data Sources 18
4.4 Data Engineering Pipeline 19
4.4.1 Ingestion and Decoupling 20
4.4.2 Document State Lifecycle 20
4.4.3 Preprocessing and Chunking 21

4.5 Embedding and Indexing . . . 21
4.6 Storage Architecture (Medallion Layout) . . . 22
4.7 Database Schema . . . 22
4.8 Prompt Construction and Grounded Generation . . . 23
4.9 LLM Backend Fallback Chain . . . 24
4.10 Deployment, Monitoring, and MLOps . . . 24
4.10.1 LLM Generation Configuration . . . 25
4.10.2 Drift Detection and Remediation . . . 26
4.10.3 Health Monitoring . . . 27
4.10.4 CI/CD and Deployment Automation . . . 27
4.11 AWS Cloud Deployment Architecture . . . 28
4.12 User Interface . . . 28

5 Experimental Plan and Evaluation Metrics 29
5.1 Evaluation Objective . . . 29
5.2 Experimental Configurations . . . 29
5.3 Evaluation Dataset . . . 30
5.4 Evaluation Metrics . . . 30
5.4.1 Context Precision . . . 30
5.4.2 Faithfulness and Hallucination Verification . . . 31
5.4.3 Latency . . . 31
5.4.4 Cost Per Query . . . 31
5.4.5 Reranking Weight Optimization . . . 31
5.4.6 Sensitivity Analysis . . . 31
5.4.7 Statistical Tests for Baseline vs. Hybrid Comparison . . . 32
5.4.8 Citation Grounding with Partial Relevance . . . 32
5.4.9 Composite Performance Metric . . . 32
5.4.10 Drift Detector Validation . . . 33
5.4.11 Monthly Cost Projection . . . 33
5.5 Analysis and Discussion Framework . . . 34
5.6 Result Visualizations . . . 35
5.6.1 Baseline vs. Hybrid Retrieval Comparison . . . 35
5.6.2 Grid Search: Precision–Latency Pareto Frontier . . . 35
5.6.3 Per-Source Retrieval Distribution . . . 36
5.6.4 Latency by Query Category . . . 37
5.6.5 Multi-Metric Radar Comparison . . . 37
5.6.6 Per-Query Latency Shift . . . 38
5.6.7 Evaluation Summary Dashboard . . . 38

6 Implementation Progress and Deviations 40

6.1 Overall Implementation Status 40
6.2 Deviations from the Original Proposal 42
6.3 Expanded Data Source Coverage 44
6.4 Enhanced Preprocessing Pipeline 44
6.5 AWS Free Tier Deployment 44
6.6 Test Coverage 45
6.7 Custom CloudWatch Metrics 47
6.8 Engineering Observations 47
6.9 System Demonstration 48
6.9.1 Frontend: Hybrid Retrieval Query with Citation-Grounded Response . . 48
6.9.2 Frontend: Baseline vs. Hybrid Comparison 48
6.9.3 Unit Test Execution 49
6.9.4 Docker Compose: Running Services 50
6.9.5 CI/CD Pipeline: GitHub Actions 51
6.9.6 Database: Populated Corpus from Five Sources 52

7 Remaining Work and Limitations 54
7.1 Remaining Work 54
7.1.1 Phase A: System Validation and Corpus Building (Mar W3 – Mar W4) . 54
7.1.2 Phase B: Evaluation and Experimentation (Mar W4 – Apr W1) 55
7.1.3 Phase C: Analysis and Monitoring Validation (Apr W1 – Apr W2) . . . 55
7.1.4 Phase D: Final Report and Submission (Apr W2) 55
7.2 Known Limitations 56
7.3 Project Timeline 57
7.4 Future Extensions 57

REFERENCES 59

A Source Code Availability 62

B System Demonstration Screenshots 63

CHAPTER 1

INTRODUCTION

1.1 Problem Description and Motivation

Motivating scenario. A machine learning engineer at a technology company needs to stay current with advances in transformer efficiency. She must manually monitor ArXiv preprints, scan Hacker News threads, and cross-reference multiple sources—a process that is time-consuming, fragmented, and prone to missing recent developments. An intelligent assistant capable of synthesizing both scholarly and community-sourced knowledge into a single, citation-grounded response would significantly accelerate her workflow. TechPulse is designed precisely for this use case.

The exponential growth of emerging technologies—including artificial intelligence, machine learning, distributed systems, and advanced computing—has produced an unprecedented volume of technical knowledge distributed across heterogeneous platforms. Scholarly repositories such as ArXiv provide structured, peer-reviewed or preprint research outputs contributing to theoretical and methodological advancement. Community-driven platforms and industry discussion forums such as Hacker News reflect near-real-time practitioner experiences and emerging tool ecosystems. However, the fragmentation of these knowledge regimes creates an information landscape that is increasingly difficult for technology professionals to synthesize effectively and in a timely manner.

Purely generative large language model (LLM) assistants are insufficient for emerging technology intelligence tasks due to their static knowledge cutoff, susceptibility to hallucinated responses, lack of verifiable evidence grounding, and inability to integrate with continuously updating data streams (Ji et al., 2023). Although Retrieval-Augmented Generation (RAG) architectures partially address these limitations by incorporating external context, baseline vector-only retrieval approaches may still retrieve semantically adjacent but contextually irrelevant documents. Such systems often fail to incorporate temporal relevance signals, structured metadata constraints, or recency-aware ranking mechanisms, which are essential for accurately capturing rapidly evolving technology trends.

Moreover, many existing RAG research prototypes focus primarily on retrieval accuracy without addressing production-readiness: deterministic pipeline design, idempotent ingestion, monitoring and observability, deployment automation, and cloud cost governance are frequently neglected. This limits their applicability in real-world operational environments.

The central engineering and research challenge addressed in this project is therefore:

How can a scalable, continuously updating hybrid retrieval system be designed and deployed to synthesize emerging technology trends from heterogeneous scholarly and industry sources while satisfying latency and cost constraints?

This project is motivated by three converging factors. First, technology professionals increasingly require timely and citation-grounded intelligence that synthesizes insights from both scholarly research and rapidly evolving industry discussions. Second, mature serverless cloud infrastructures now enable cost-effective deployment of production-grade data and machine learning pipelines. Third, a gap remains in the Retrieval-Augmented Generation (RAG) literature between retrieval-focused research prototypes and operationally robust systems capable of continuous ingestion, evaluation, and deployment in real-world environments.

1.2 Research Questions

To address the central problem, this study investigates the following research questions:

1. **Data Engineering.** How can a cloud-native pipeline be designed to reliably ingest and preprocess heterogeneous scholarly and near-real-time industry data in a deterministic and idempotent manner?
2. **Retrieval Optimization.** Does hybrid retrieval—combining metadata filtering, vector similarity search, and lightweight reranking—improve retrieval precision and grounding faithfulness compared with baseline vector-only retrieval?
3. **Operational Trade-offs.** What trade-offs arise between retrieval precision, p95 latency, and cost per query under realistic workload conditions?
4. **MLOps Integration.** How can deployment, monitoring, version control, and cost governance be integrated to ensure the reliability and maintainability of a production-grade RAG system?

1.3 Objectives of the Study

The primary objective is to design, implement, and evaluate a hybrid Retrieval-Augmented Generation system capable of synthesizing real-time emerging technology intelligence within a scalable, cost-aware cloud-native architecture. The specific objectives are:

1. To construct a cloud-native data engineering pipeline for batch and near-real-time ingestion from five heterogeneous data sources.
2. To implement and compare baseline vector-only retrieval against hybrid retrieval with metadata filtering and reranking-lite optimization.
3. To evaluate system performance across context precision, faithfulness, p95 latency, and cost per query.

4. To deploy and monitor the system using serverless AWS infrastructure consistent with MLOps best practices (Sculley et al., 2015; Breck et al., 2017).

1.4 Significance of the Study

This project contributes at the intersection of data engineering, natural language processing, and MLOps. From a technical perspective, it demonstrates how retrieval optimization can be integrated within a production-grade cloud architecture without relying on computationally expensive cross-encoder rerankers or specialized vector database infrastructure. The system provides a practical framework for grounded intelligence synthesis across academic and industry domains while maintaining architectural simplicity suitable for scalable deployment. From a practical perspective, the resulting system offers a blueprint for citation-supported technology intelligence synthesis that is operationally maintainable, cost-governed, and deployable within resource-constrained environments such as the AWS Free Tier.

1.5 Alignment Between Research Questions and Evaluation

Table 1.1 provides an explicit mapping from each research question to the corresponding evaluation metric and experimental configuration used to answer it.

Table 1.1. Mapping of Research Questions to Evaluation Metrics and Methodology.

RQ	Question (abbreviated)	Primary Metric	Evaluation Approach
1	Cloud-native deterministic ingestion	Pipeline idempotency; SHA-256 deduplication pass rate	Unit tests (24 ingestion tests); repeated pipeline runs
2	Hybrid vs. baseline retrieval precision	Context precision (RAGAS); faithfulness ≥ 0.80; citation grounding > 0.80	Paired comparison over 50 queries; Wilcoxon signed-rank test; Cohen's d; bootstrap 95% CI
3	Latency and cost trade-offs	p95 latency $<$ 2s; cost $<$ USD 15/month	Per-query timing; token usage profiling; grid search Pareto frontier
4	MLOps integration	Drift detection coverage; CI/CD pass rate; health-check uptime	Drift simulation runs; CloudWatch alarm verification

The remainder of this report is organized as follows: Chapter 2 reviews the relevant literature;

Chapter 3 presents the methodology and hypotheses; Chapter 4 describes the system architecture; Chapter 5 defines the evaluation framework; Chapter 6 documents implementation status and deviations; and Chapter 7 outlines remaining work and limitations. The following chapter reviews the technical foundations underpinning these design decisions.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020) as a framework for augmenting sequence-to-sequence generation with a dense retrieval component, enabling the model to access external documents at inference time. This approach significantly improves performance on knowledge-intensive tasks by conditioning generation on retrieved evidence rather than relying solely on parametric memory. Despite their scale, LLMs remain prone to hallucination when responding to domain-specific or time-sensitive queries (Ji et al., 2023), making retrieval-based augmentation a practically important technique.

Recent surveys (Gao et al., 2023; Zhang et al., 2024) categorize RAG systems into four architectural types: (1) single-stage dense retrieval; (2) hybrid lexical–semantic retrieval; (3) multi-stage retrieval with reranking; and (4) iterative or self-reflective retrieval. While baseline dense retrieval improves factual grounding, empirical benchmarks such as BEIR (Thakur et al., 2021) demonstrate that semantic similarity alone is insufficient for high-precision retrieval in dynamic or heterogeneous corpora. Foundational retrieval-augmented systems such as Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) demonstrate retrieval augmentation at scale but require computationally intensive pretraining pipelines unsuited to cost-constrained production environments.

2.2 Embedding Models and Vector Search

Dense retrieval relies on embedding models that transform text into high-dimensional vector representations. Sentence-transformer architectures such as MiniLM (Wang et al., 2020) provide computationally efficient embeddings with favorable trade-offs between semantic expressiveness and inference latency. PostgreSQL with the pgvector extension enables cosine similarity search alongside SQL-based metadata filtering within a single relational storage layer, reducing architectural complexity compared to dedicated vector databases such as FAISS or Milvus. This integrated approach is increasingly favored for production RAG deployments (Gao et al., 2023).

2.3 Hybrid Retrieval and Reranking

Hybrid retrieval combines lexical matching (e.g., BM25 or keyword filtering) with semantic similarity search to improve retrieval precision. Thakur et al. (2021) demonstrate that hybrid approaches consistently outperform purely dense retrieval across heterogeneous benchmarks. Reranking further refines candidate relevance using additional contextual signals. Cross-encoder

rerankers achieve high accuracy but incur substantial computational overhead (Nogueira & Cho, 2019); lightweight multi-factor reranking—combining similarity score, recency weighting, and keyword overlap—offers comparable grounding improvements at lower latency (Gao et al., 2023). In dynamic knowledge domains, recency-aware retrieval is particularly important to avoid outdated context injection.

2.4 RAG Evaluation Methodologies

Evaluating RAG systems requires assessing both retrieval quality and generation faithfulness simultaneously. Es et al. (2023) introduced RAGAS, a reference-free evaluation framework that measures faithfulness, answer relevance, context precision, and context recall using automated LLM-based scoring, enabling scalable evaluation without large human annotation pools. Complementary work by Saad-Falcon et al. (2023) proposed ARES, which uses a small set of human-annotated examples to train lightweight judges capable of scoring RAG systems across multiple quality dimensions. These frameworks demonstrate that even compact evaluation sets—in the range of 20 to 50 annotated examples—can yield statistically meaningful comparisons between retrieval configurations when evaluation dimensions are clearly scoped and metrics are carefully defined (Es et al., 2023). This finding provides empirical support for the evaluation scale adopted in TechPulse.

2.5 Data and Concept Drift in Retrieval Systems

In dynamic knowledge domains, retrieval system performance can degrade over time due to data drift—a shift in the statistical properties of incoming documents relative to the indexed corpus—or concept drift, where the semantic meaning of user queries evolves independently of corpus updates (Gama et al., 2014). Lu et al. (2018) provide a comprehensive taxonomy of drift detection methods, categorizing approaches into statistical process control, windowed distribution comparison, and ensemble-based disagreement detection. In retrieval-augmented systems specifically, drift can manifest as declining context precision when newly ingested documents diverge from the embedding space learned during the original indexing phase. Addressing drift requires both monitoring mechanisms that can detect distributional shifts in retrieval scores over time, and remediation actions such as corpus re-indexing, embedding model refresh, or query routing adjustments (Bayram et al., 2022). TechPulse incorporates a dual-criteria embedding distribution monitoring strategy informed by these findings.

2.6 Hallucination Detection and Faithfulness Verification

Despite retrieval augmentation, generative models can still produce hallucinated content by inferring beyond the supplied context, over-generalizing retrieved passages, or introducing spurious cross-document associations (Ji et al., 2023). Manakul et al. (2023) proposed SelfCheckGPT, a consistency-based approach that detects hallucinations by sampling multiple outputs and measuring inter-sample consistency—a technique applicable without access to ground-truth

references. Min et al. (2023) proposed FActScore, which decomposes generated responses into atomic claims and individually verifies each claim against a reference corpus, offering fine-grained faithfulness assessment. At a system design level, Shi et al. (2023) demonstrate that irrelevant context injected alongside relevant passages can distract generative models and induce hallucinated responses even when correct information is present—motivating strict top-k selection and high-precision retrieval as the primary anti-hallucination mechanism in TechPulse.

2.7 MLOps and Production Machine Learning Systems

Sculley et al. (2015) identify hidden technical debt as a central challenge in ML systems, emphasizing monitoring, reproducibility, and lifecycle management. MLOps extends DevOps principles to ML workflows, incorporating continuous integration, deployment automation, model versioning, and observability (Breck et al., 2017). Cloud-native, serverless architectures enable scalable and maintainable deployment with cost governance (Kreuzberger et al., 2023; AWS, 2023). While RAG research frequently focuses on retrieval effectiveness, production-readiness—including pipeline idempotency, schema versioning, and budget-aware inference—remains underexplored.

2.8 Research Gap

The reviewed literature establishes six key insights: (1) RAG improves factual grounding but baseline dense retrieval may lack precision; (2) hybrid retrieval outperforms vector-only search in heterogeneous settings; (3) lightweight reranking can improve relevance without prohibitive overhead; (4) production ML systems require robust MLOps practices beyond model accuracy; (5) even compact evaluation sets of 20–50 annotated examples can yield statistically meaningful RAG comparisons when evaluation dimensions are clearly scoped; and (6) hallucination in RAG systems can persist despite retrieval augmentation, necessitating explicit faithfulness verification mechanisms.

Critically, existing hybrid RAG systems either address a subset of these concerns or are evaluated in static, single-source corpora that do not reflect the temporal dynamics of multi-source technology intelligence. Atlas (Izacard et al., 2022) and RETRO (Borgeaud et al., 2022) operate at scale but require pretraining pipelines and do not address operational cost governance or retrieval-quality drift. Production systems such as Perplexity AI and Bing Copilot incorporate real-time retrieval but do not expose evaluation methodology or reranking designs. Academic RAG prototypes frequently omit monitoring, idempotent ingestion, and budget constraints that are essential for sustained deployment.

Closest prior systems. Several open-source RAG frameworks warrant direct comparison. Haystack (deepset, 2023) provides a modular pipeline framework supporting hybrid retrieval with Elasticsearch BM25 and dense retrievers, but delegates MLOps, drift detection, and cost governance to external tooling. LlamaIndex (Liu, 2022) offers composable retrieval abstractions

including metadata filtering and reranking; however, it is a library rather than a deployed system and does not prescribe ingestion scheduling, monitoring infrastructure, or budget enforcement. Verba (Weaviate, 2023) implements a RAG frontend with Weaviate vector search but uses single-source ingestion and omits reranking, drift monitoring, and hallucination verification. STORM (Shao et al., 2024) automates long-form article generation with multi-source retrieval but targets offline synthesis rather than interactive query answering and does not address operational constraints. LangChain (LangChain, 2023) provides composable chain abstractions with optional hybrid retrieval and agent tooling, but treats MLOps, cost governance, and drift detection as external concerns. Production systems such as Perplexity AI and Bing Copilot incorporate real-time retrieval and reranking at scale but do not expose their evaluation methodology, reranking designs, or cost governance strategies, making direct dimensional comparison infeasible. Each of these systems addresses a different subset of the six dimensions; to our knowledge, none integrates all six—hybrid retrieval with lightweight reranking, real-time multi-source ingestion, cost governance, MLOps monitoring, and explicit hallucination verification—within a single cost-governed, continuously monitored, cloud-native architecture. TechPulse's contribution is best understood as a *systems integration* effort: the individual techniques are established, but their combination within a student-deployable, budget-constrained prototype addresses a practical gap not covered by existing frameworks or research prototypes.

Table 2.1 summarizes representative related work and positions TechPulse relative to prior systems.

Table 2.1. Comparison of Related Work and Positioning of TechPulse.

System	Retrieval	Rerank	Real-Time	Cost Gov.	MLOps	Drift Det.	Halluc. Verif.
RAG (Lewis et al., 2020)	Dense	No	No	No	No	No	No
Atlas (Izacard et al., 2022)	Dense	No	No	No	No	No	No
RETRO (Borgeaud et al., 2022)	Dense	No	No	No	No	No	No
Haystack (deepset, 2023)	Hybrid	Optional	Plugin	No	Partial	No	No
LlamaIndex (Liu, 2022)	Hybrid	Yes	No	No	No	No	No
Verba (Weaviate, 2023)	Dense	No	Single	No	No	No	No
STORM (Shao et al., 2024)	Dense	No	Multi	No	No	No	No
LangChain (LangChain, 2023)	Hybrid	Optional	Plugin	No	Partial	No	No
Typical RAG Prototype	Dense	Optional	Rare	No	Minimal	Rare	Rare
TechPulse (ours)	**Hybrid**	**Yes**	**Yes**	**Yes**	**Full**[†]	**Dual**	**3-Layer**

[†]Full MLOps is implemented and under cloud validation; evaluation results are pending Phase B execution.

In summary, the literature confirms that hybrid retrieval with lightweight reranking and production-grade MLOps integration remains underexplored for dynamic technology intelligence. While individual techniques are well established, to our knowledge no reviewed system combines all six identified dimensions within a single cost-governed, student-deployable cloud-native prototype. TechPulse addresses this integration gap. The remainder of this chapter's findings inform the methodology and technical design decisions presented in Chapter 3.

CHAPTER 3

METHODOLOGY

3.1 Overview of Technical Approach

TechPulse has been implemented as a cloud-native Hybrid Retrieval-Augmented Generation system designed to synthesize emerging technology intelligence from heterogeneous sources. The technical approach is organized around five interconnected components: (1) continuous heterogeneous data ingestion from five sources; (2) deterministic preprocessing and semantic embedding; (3) hybrid multi-stage retrieval; (4) context-bound grounded generation with automatic LLM fallback; and (5) production-grade MLOps deployment and monitoring.

The core hypotheses under evaluation are stated formally as:

- $\mathbf{H_0}$: There is no statistically significant difference in mean context precision between hybrid and baseline (vector-only) retrieval, evaluated over 50 matched queries under identical corpus and embedding conditions.
- $\mathbf{H_1}$: Hybrid retrieval achieves statistically higher mean context precision than baseline retrieval (one-sided Wilcoxon signed-rank test, $\alpha = 0.05$).

A secondary performance hypothesis requires that the precision improvement does not exceed the p95 latency bound ($\tau = 2.0$ seconds for the retrieval stage) or the monthly cost ceiling (USD 15 at an assumed workload of 1,500 queries/month, i.e., USD 0.01 per query). These hypotheses are tested through a controlled experiment comparing both configurations under identical corpus and embedding conditions, with full results reported in the final submission.

3.2 Why This Approach is Suitable

Three considerations motivate the hybrid RAG approach for this specific problem domain.

Domain characteristics. Emerging technology knowledge is temporally dynamic and distributed across heterogeneous sources. Vector-only retrieval treats all documents equally regardless of recency or source authority; hybrid retrieval with recency-aware reranking explicitly addresses temporal drift, which is critical in rapidly evolving AI/ML domains where a paper from 18 months ago may already be superseded.

Operational constraints. The system must operate within cost and latency bounds suitable for a research prototype deployed on the AWS Free Tier. The lightweight linear reranking function (Equation 3.2) adds $O(k)$ computation over the top-N filtered candidates—negligible compared to the $O(Nd)$ baseline search—and avoids the latency penalties of cross-encoder rerankers, which typically require a forward pass through a BERT-class model per candidate.

Production feasibility. The architecture uses AWS-managed services—Lambda, RDS PostgreSQL, API

Gateway, EventBridge—eliminating the need to manage dedicated compute clusters or self-hosted vector databases. This enables a student team to deploy, iterate, and monitor a production-grade system within the course timeline and a prototype budget envelope.

3.3 Technical Contributions and Design Decisions

The engineering contribution of TechPulse lies not in any single component but in their integration into a coherent, cost-governed MLOps system. Specifically, four design decisions represent well-motivated engineering choices that distinguish this system from typical RAG prototypes.

Recency-aware hybrid reranking for dynamic knowledge domains. The weighted linear reranking function (Equation 3.2) fuses semantic similarity, lexical keyword overlap, and exponential temporal decay into a single interpretable score. Unlike cross-encoder rerankers that are domain-agnostic and computationally expensive, this formulation is specifically designed for domains where content freshness is a first-class retrieval signal. The exponential decay parameter λ provides a principled, tunable mechanism for controlling how rapidly older documents lose retrieval priority.

Unified vector–relational retrieval without dedicated infrastructure. By extending PostgreSQL with pgvector, TechPulse achieves hybrid retrieval (structured SQL metadata filtering + cosine vector search) within a single managed database, eliminating the operational complexity of running a separate vector store. This design choice reduces architectural surface area, simplifies deployment automation, and enables cost-governed scaling through standard RDS instances.

Idempotent ingestion pipeline with SHA-256 content-addressed deduplication. The SHA-256 hashing strategy applied to canonicalized document fields ensures that the pipeline can be re-run safely at any time without creating duplicate records. This often-overlooked but practically important engineering practice enables safe experiment replay, crash recovery, and corpus versioning without requiring a separate deduplication service.

Budget-aware inference with programmatic halt. The integration of AWS Cost Explorer queries into the inference path—where invocations are halted programmatically when monthly cost thresholds are approached—represents an operational optimization strategy not typically addressed in RAG research prototypes. This transforms cost from a passive monitoring concern into an active constraint enforced at runtime, consistent with production-grade MLOps principles.

3.4 Baseline Retrieval Configuration

The baseline configuration performs dense cosine similarity search across the full indexed corpus. Given a query embedding $\mathbf{e}_q$ and document embeddings $\mathbf{e}_i$, cosine similarity is computed as:

$$\mathrm{Sim}(q,i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\|\|\mathbf{e}_i\|} \tag{3.1}$$

Top-N documents are selected purely on similarity ranking with a minimum similarity threshold of 0.15 to filter retrieval noise. This low threshold is intentionally permissive in the baseline to avoid artificially

handicapping it; borderline documents that would be excluded by a stricter threshold are instead ranked low by the subsequent reranking stage in the hybrid configuration. Under exact search, computational complexity scales as $O(Nd)$, where N is the number of indexed chunks and $d = 384$ (the MiniLM embedding dimension). This configuration serves as the experimental control for retrieval performance comparison.

3.5 Implemented Hybrid Retrieval Configuration

The hybrid configuration introduces three sequential refinement stages applied before generation.

Stage 1 — Metadata Filtering. Structured SQL constraints reduce the candidate space based on document recency (a 180-day sliding window), source type, and document state. Only documents in the `INDEXED` state are considered. The 180-day window was chosen to balance corpus recency with sufficient document volume: at the expected ingestion rates across five sources, the window is projected to yield 500–2,000 indexed chunks, providing a meaningful retrieval pool while excluding content more than six months old, which is considered stale for fast-moving AI/ML topics. This pre-filtering stage reduces retrieval noise prior to semantic ranking.

Robustness to missing and noisy metadata. Because the five ingestion sources vary in metadata completeness, the filter is designed to degrade gracefully. The recency filter uses an inclusive `OR` clause (`published_at IS NULL OR published_at >= threshold`), so documents with missing publication dates are retained rather than silently dropped—this is particularly relevant for Hacker News "Ask HN" and "Show HN" self-posts, which occasionally lack external URLs or have approximate timestamps. During ingestion, each source adapter normalizes metadata before insertion: titles and content are whitespace-normalized; empty URLs default to the source platform's item permalink; and missing timestamps are coerced from the API's Unix epoch field or, for RSS feeds, from the `<pubDate>` element with a fallback to ingestion time. The reranking stage further mitigates noisy metadata: documents with `NULL` publication dates receive a recency weight of 0 (Equation 3.3), meaning they can still be retrieved via semantic similarity but are deprioritized by the temporal signal. This design ensures that incomplete metadata degrades relevance ranking gracefully rather than causing silent retrieval failures.

Stage 2 — Vector Similarity Search. Cosine similarity (Equation 3.1) is applied over the filtered candidate subset using the pgvector `<=>` operator. To provide sufficient candidates for reranking, $4 \times$ top_k results are retrieved at this stage. The $4\times$ multiplier follows the oversampling convention used in cross-encoder reranking pipelines (Nogueira & Cho, 2019), giving the reranker enough diversity to meaningfully reorder results without expanding the candidate pool to the point where reranking overhead dominates latency.

Stage 3 — Reranking-Lite. A lightweight weighted scoring function refines candidate ordering:

$$\text{Score}_i = \alpha \cdot \text{Sim}_i + \beta \cdot \text{KeywordOverlap}_i + \gamma \cdot \text{RecencyWeight}_i \tag{3.2}$$

where $\alpha + \beta + \gamma = 1$. The implemented default weights are $\alpha = 0.6$, $\beta = 0.2$, $\gamma = 0.2$, configurable via environment variables (`RERANK_ALPHA`, `RERANK_BETA`, `RERANK_GAMMA`) to support grid search during

evaluation.

Recency is modeled using exponential decay:

$$\text{RecencyWeight}_i = e^{-\lambda \Delta t_i} \tag{3.3}$$

where Δt_i is document age in days and $\lambda = 0.01$ (configurable via `RECENCY_LAMBDA`). At this decay rate, a document published 69 days ago retains $e^{-0.69} \approx 0.50$ of its recency weight (the "half-life" of relevance $\approx$ 69 days, or approximately 2.3 months), and a document at the 180-day filter boundary retains $e^{-1.8} \approx 0.17$. This profile reflects the observation that AI/ML content has a practical relevance half-life of approximately 2–3 months; λ is configurable to enable sensitivity analysis during evaluation.

Figure 3.1 illustrates the conceptual difference between the baseline and hybrid retrieval pathways.

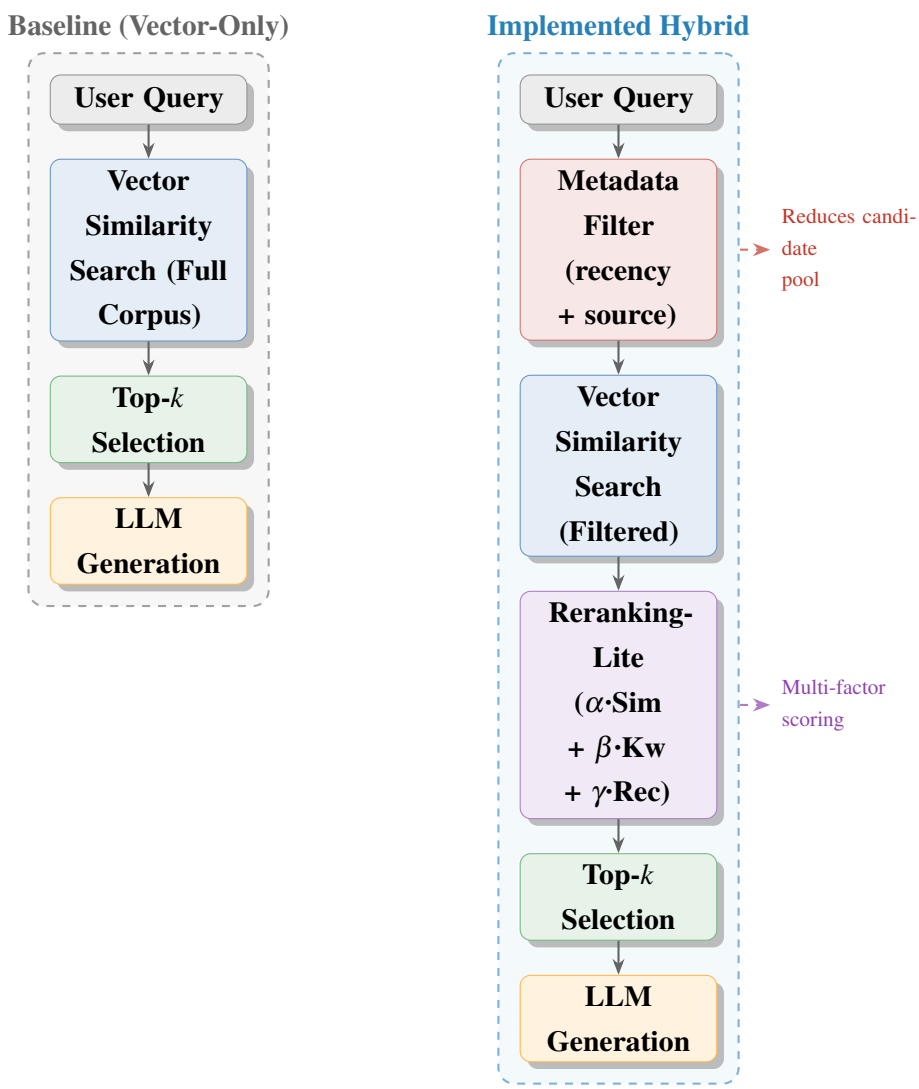

Figure 3.1. Baseline vector-only retrieval (left) vs. implemented hybrid retrieval pipeline (right).

3.6 Hallucination Verification Strategy

TechPulse applies a three-layer verification approach to detect and flag potentially hallucinated outputs. The prompt construction and template details are presented in Section 4.8.

1. **Prompt layer.** The system instruction enforces strict evidence grounding and abstention on insufficient context.
2. **Evaluation layer.** RAGAS faithfulness scoring decomposes each generated response into atomic claims and verifies whether each claim is supported by at least one retrieved document.
3. **Output layer — Citation traceability.** At runtime, automated citation grounding verification analyzes each response sentence-by-sentence to compute a citation ratio—the fraction of sentences containing valid `[Source N]` references. Responses falling below the configurable citation grounding threshold are flagged as potentially hallucinated and replaced with an abstention message, with the original response preserved in metadata for review.

3.7 Optimization Objective

The retrieval task is framed as a constrained optimization problem:

$$\max \text{ Precision}(q) \quad \text{subject to} \quad \text{Latency} \leq \tau \tag{3.4}$$

where τ is the p95 latency target of 2 seconds. This formulation makes the precision–latency trade-off explicit and provides a principled basis for tuning reranking depth and metadata filter selectivity.

3.8 Tool Selection and Implementation Stack

Table 3.1 summarizes the implemented tools and their justification. The stack has been adapted from the original proposal to operate within AWS Free Tier constraints, as detailed in Chapter 6.

Table 3.1. Implemented Tools, Services, and Justification.

Component	Tool / Service	Justification
Ingestion orchestration	AWS Lambda + EventBridge	Serverless, scheduled/event-driven, no cluster management
Message queue	Amazon SQS + DLQ	Decouples ingestion from embedding; failure resilience
Raw storage	Amazon S3 (medallion layout)	Cost-effective, durable, prefix-based lineage tracking
Embedding model	MiniLM (sentence-transformers)	Low latency, 384-dim, serverless-compatible
Vector + relational store	RDS PostgreSQL + pgvector	Unified hybrid retrieval; AWS Free Tier compatible
Generative model	Groq (Llama 3.1 8B Instant, primary); Bedrock (Amazon Nova Micro, fallback); Ollama / HuggingFace (local fallbacks)	Auto-cascade fallback chain; Groq free-tier API available across all environments
Frontend	React (S3 static hosting)	Lightweight, decoupled from backend
API routing	Amazon API Gateway (HTTP)	Managed, HTTPS, CORS support
CI/CD	GitHub Actions + SAM	Reproducible IaC deployments, automated testing
Monitoring	CloudWatch + SNS alarms	Observability, error alerts, health checks
Evaluation	RAGAS + grid search	Standardized RAG quality metrics + weight optimization

3.9 System Context Overview

Figure 3.2 presents a high-level context view of TechPulse, showing external actors, data flows, and system boundaries before the detailed architecture is described in Chapter 4.

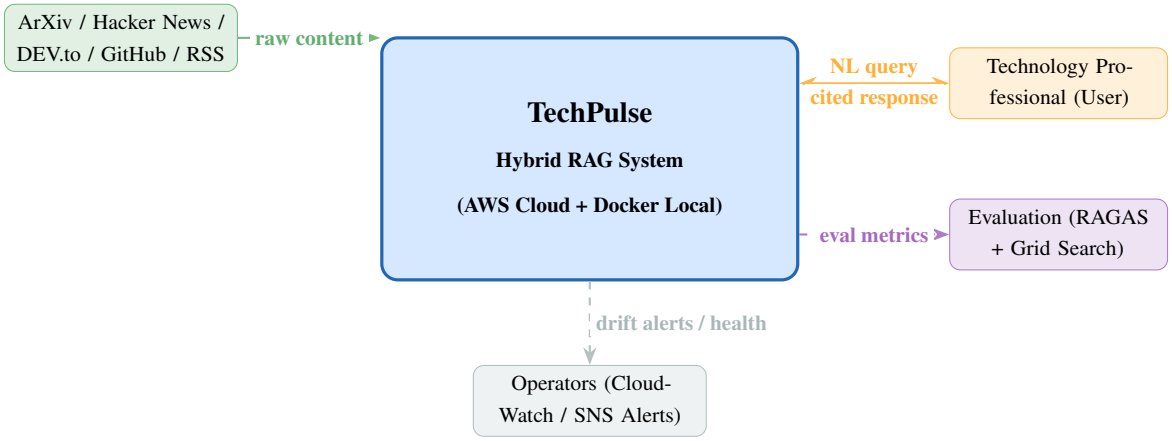

Figure 3.2. System context diagram for TechPulse. Solid arrows represent data and query flows; dashed arrows represent monitoring and evaluation signals.

The remainder of this report translates these methodological decisions into a concrete system architecture (Chapter 4), evaluation framework (Chapter 5), implementation status (Chapter 6), and remaining work (Chapter 7).

CHAPTER 4

SYSTEM ARCHITECTURE AND PIPELINE DESIGN

4.1 Architectural Overview

TechPulse has been implemented as a six-layer, cloud-native system deployed on the AWS Free Tier. Each layer is logically independent with well-defined interfaces, enabling modular development, testing, and future extension. The architecture was adapted from the original proposal to operate within Free Tier cost and service constraints, replacing Aurora Serverless with standard RDS PostgreSQL and favouring free-tier LLM APIs while preserving the core hybrid retrieval and MLOps capabilities.

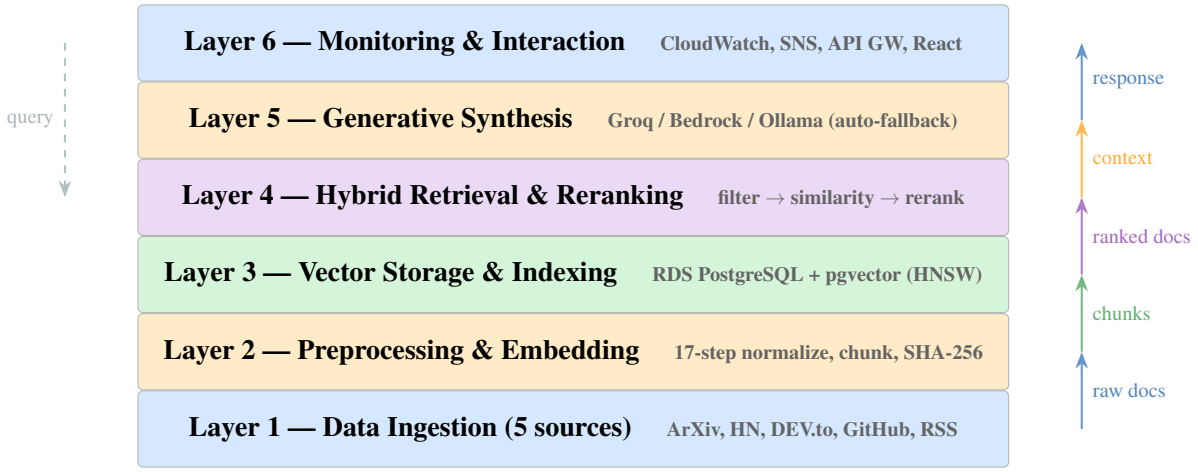

Figure 4.1. Six-layer conceptual architecture of TechPulse. Right-hand arrows show the upward data flow; the left-hand dashed arrow shows the query path.

4.2 Alignment with the AWS Well-Architected Framework

The TechPulse architecture is designed in explicit alignment with the AWS Well-Architected Framework (AWS, 2023). Table 4.1 summarises how each pillar is addressed within the implemented system.

Table 4.1. TechPulse Alignment with the AWS Well-Architected Framework Six Pillars.

Pillar	TechPulse Implementation
Operational Excellence	Automated ingestion via EventBridge (6-hour cycle); CI/CD pipeline using GitHub Actions and SAM; CloudWatch logging, custom metrics, and health-check Lambda (5-minute interval)
Security	Controlled API access via Amazon API Gateway with CORS; rate limiting via slowapi; all services managed within AWS default VPC; no direct model invocation from the frontend
Reliability	Decoupled ingestion and embedding pipelines via SQS; Dead Letter Queue for failure recovery and replay with 14-day retention; bounded retry logic with exponential backoff for LLM invocations; automatic LLM backend fallback chain
Performance Efficiency	Serverless Lambda scaling; lightweight MiniLM embedding model (384-dim); metadata pre-filtering reduces vector search scope; HNSW index for approximate nearest-neighbor search
Cost Optimization	Serverless-only execution (pay-per-invocation); programmatic budget guard via Cost Explorer; Groq free-tier inference as primary backend (500k tokens/day); Amazon Bedrock available in Free Tier; db.t3.micro RDS instance
Sustainability	Event-driven processing avoids idle compute; SQS decoupling prevents over-provisioning; right-sized Lambda memory allocations (512MB–1024MB)

4.3 Data Sources

The original proposal specified two data sources (ArXiv and Hacker News). During implementation, the data source coverage was expanded to five heterogeneous sources to improve corpus diversity, topical breadth, and temporal coverage. Table 4.2 summarizes the implemented data sources.

Table 4.2. Implemented Data Sources and Roles.

Source	Type	Role	Frequency
ArXiv API	Batch	Scholarly grounding corpus (cs.AI, cs.LG, cs.CL, cs.IR, cs.SE)	Every 12 hours
Hacker News API	Near-real-time	Community-curated tech discussions	Every 30 minutes
DEV.to API	Near-real-time	Practitioner tutorials and industry articles (21 tags)	Every 30 minutes
GitHub Trending	Batch	Open-source project intelligence (10 topic queries)	Every 30 minutes
RSS/Atom Feeds	Batch	Curated technology journalism (10 feeds)	Every 30 minutes

The frequencies in Table 4.2 reflect the local scheduler (`scheduler.py`), which provides fine-grained per-source intervals. On AWS, a single EventBridge rule triggers the ingestion Lambda every 6 hours, executing all five sources in one invocation.

The combination provides complementary temporal and authority profiles: ArXiv contributes rigorously authored research with stable identifiers; Hacker News contributes practitioner signals and community-curated trending discussions; DEV.to provides developer-focused tutorials and implementation insights; GitHub Trending captures emerging open-source tools and frameworks; and curated RSS feeds from outlets including Ars Technica, TechCrunch, Wired, IEEE Spectrum, and MIT News provide technology journalism coverage.

4.4 Data Engineering Pipeline

The pipeline has been implemented around four design principles: determinism (identical input produces identical output), idempotency (re-running does not create duplicate records), observability (all state transitions are logged), and failure resilience (failures are isolated and recoverable).

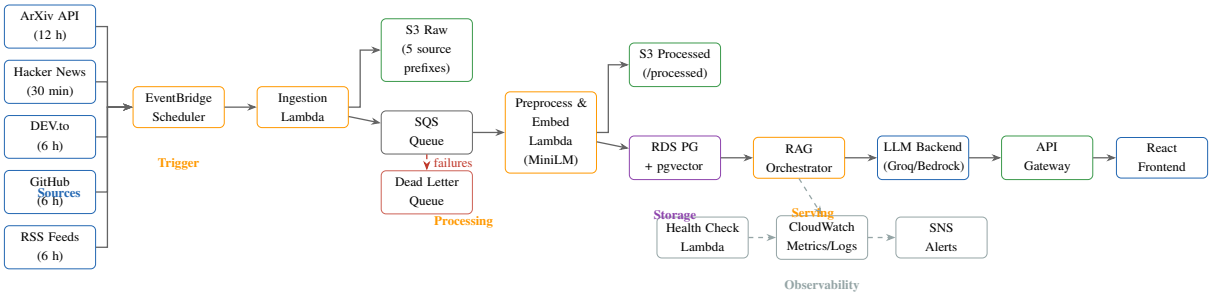

Figure 4.2. End-to-end data engineering and serving pipeline. Solid arrows indicate data flow; dashed arrows indicate monitoring signals.

4.4.1 Ingestion and Decoupling

Ingestion has been implemented through AWS Lambda functions orchestrated by Amazon EventBridge, supporting both scheduled and event-driven execution. Each of the five ingesters (ArXiv, Hacker News, DEV.to, GitHub, RSS) operates independently with source-specific data extraction logic. Raw artifacts are persisted in Amazon S3 under a structured prefix hierarchy (`/raw/arxiv/`, `/raw/hn/`, `/raw/devto/`, `/raw/github/`, `/raw/rss/`, `/processed/`) to preserve transformation lineage. The ingestion pipeline is decoupled from the embedding pipeline via Amazon SQS, providing backpressure control, failure isolation, and independent horizontal scaling. Messages that fail processing after three retry attempts are routed to a Dead Letter Queue with 14-day retention for controlled replay.

All five ingesters share a common architectural pattern: (1) fetch data from API or feed; (2) extract and normalize fields; (3) compute SHA-256 content hash over canonicalized fields; (4) insert into the `documents` table with `ON CONFLICT (content_hash) DO NOTHING` for idempotent deduplication; (5) write raw JSON to S3; and (6) enqueue document ID to SQS for downstream processing.

4.4.2 Document State Lifecycle

Each document progresses through a four-state deterministic lifecycle to guarantee idempotency and traceability. Table 4.3 defines these states.

Table 4.3. Document State Transition Model.

State	Description	Storage	Trigger
RAW	Document ingested from API	Amazon S3	Successful API fetch
PROCESSED	Cleaned, normalized, chunked	S3 (`/processed`)	Preprocessing complete
EMBEDDED	Vector representation generated	RDS (pgvector)	Embedding computed
INDEXED	Available for retrieval	RDS metadata table	Vector committed

Figure 4.3 illustrates the document state machine, showing the trigger and storage destination for each transition.

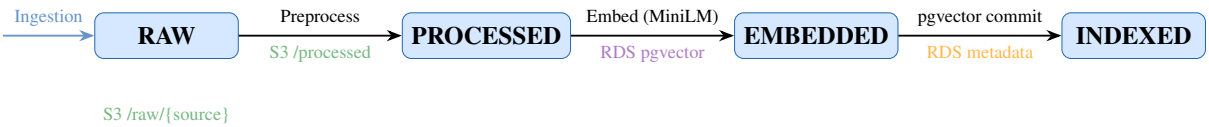

Figure 4.3. Document state machine with four deterministic states. Transition labels show triggering processes (top) and storage destinations (bottom).

To guarantee idempotency, canonicalized content fields are hashed using SHA-256 before processing.

The hash covers source-specific canonical fields: for ArXiv, title, abstract, and publication date; for Hacker News, title, URL, and timestamp; for DEV.to, article title, URL, and tags; for GitHub, repository full name, description, and stars; for RSS, entry title, link, and published date. Duplicate hashes are skipped at ingestion via PostgreSQL `ON CONFLICT` clauses, ensuring consistent corpus integrity across experimental runs.

4.4.3 Preprocessing and Chunking

Textual preprocessing has been implemented as a comprehensive 17-step normalization pipeline in the `normalize_text()` function. Table 4.4 summarises the grouped operations applied in sequence.

Table 4.4. Preprocessing Pipeline: 17-Step Text Normalization.

Category	Steps	Operations
HTML Cleanup	1–3	Entity unescaping, comment removal, tag stripping
Code & Markdown	4–6	Fenced/inline code removal, image/link normalization
URL Removal	7	HTTP/HTTPS/FTP URL stripping
Formatting Cleanup	8–12	Heading, bold/italic, strikethrough, rule, blockquote markers
Unicode Normalization	13–14	Emoji/pictographic removal (6 Unicode ranges), control characters
Whitespace Normalization	15–17	Punctuation collapse, NBSP normalization, whitespace trimming

Following normalization, documents are segmented using token-based chunking with the `tiktoken` tokenizer (`cl100k_base` encoding). The implemented configuration uses a 500-token window with 50-token overlap. The 500-token window was selected to fit comfortably within the top-k context budget: with $k = 5$ chunks, the total retrieved context consumes approximately 2,500 tokens—well within the 128k-token window of Groq-hosted Llama 3.1 8B Instant and the 128k-token window of Amazon Nova Micro (Bedrock), while providing sufficient per-chunk semantic coherence. The 50-token (10%) overlap prevents important information at chunk boundaries from being split across non-adjacent retrieval candidates.

4.5 Embedding and Indexing

Semantic representations are generated using the `all-MiniLM-L6-v2` sentence-transformer model (Wang et al., 2020), producing 384-dimensional embeddings. The model is loaded as a lazy-initialized singleton to minimize memory overhead under serverless execution constraints. Embeddings are stored in RDS PostgreSQL using the pgvector extension with an HNSW (Hierarchical Navigable Small World) index configured with $m = 16$ and ef_construction $= 64$, enabling efficient approximate nearest-neighbor

search with cosine distance. The parameters $m = 16$ (maximum number of bidirectional links per node) and ef_construction $= 64$ (size of the dynamic candidate list during index construction) follow the pgvector recommended defaults for datasets up to $\sim$100k vectors, providing a favourable balance between index build time, memory footprint, and recall (pgvector documentation recommends $m \in [8, 64]$ and ef_construction $\geq 2 \times m$).

4.6 Storage Architecture (Medallion Layout)

Table 4.5. Storage Layers and Responsibilities (Medallion Architecture).

Layer	Technology	Purpose
Raw (Bronze)	Amazon S3 (`/raw/{source}/`)	Store unprocessed API responses
Processed (Silver)	Amazon S3 (`/processed/`)	Store cleaned and chunked documents
Vector (Gold)	RDS PostgreSQL + pgvector	Store embeddings and structured metadata

4.7 Database Schema

The PostgreSQL database schema implements three core tables. The `documents` table stores ingested records with fields for source, title, content, URL, publication date, SHA-256 content hash (unique constraint), document state, and timestamps. The `chunks` table stores preprocessed text segments with their 384-dimensional vector embeddings, linked to parent documents via foreign key. The `drift_baselines` table persists drift detection history including mean similarity, standard deviation, probe count, and alert status.

Four database indexes support efficient retrieval: an HNSW vector index on chunk embeddings (`vector_cosine_ops`, $m = 16$, ef_construction $= 64$), and B-tree indexes on `published_at` (descending), `source`, and `state` for metadata filtering performance.

Figure 4.4 presents the entity–relationship diagram of the PostgreSQL schema.

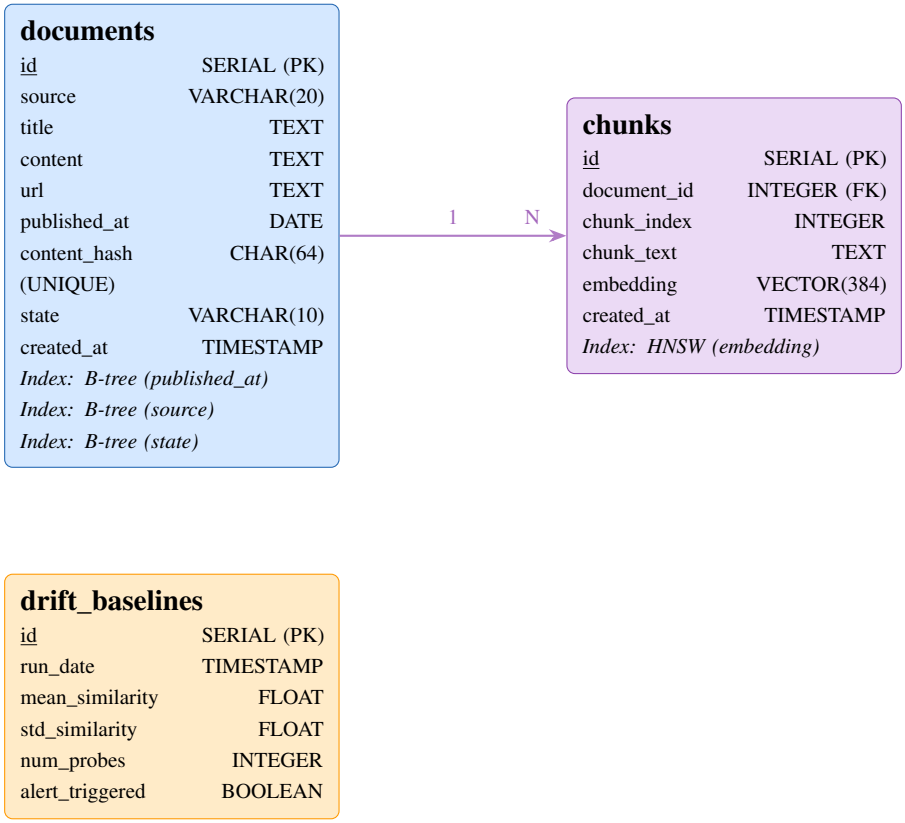

Figure 4.4. Entity–relationship diagram of the TechPulse PostgreSQL schema.

4.8 Prompt Construction and Grounded Generation

Retrieved top-k documents are concatenated into a structured prompt template with four logical sections: (1) **System Instruction**—defines assistant behavior, enforces evidence-grounded responses, and mandates `[Source N]` citation format; (2) **Context Block**—concatenates retrieved document excerpts with source identifiers, titles, and publication dates; (3) **User Question**—the original user query; and (4) **Citation Instruction**—requires the model to reference supporting documents.

Listing 4.1: Implemented Structured Generation Prompt Template.

```
SYSTEM:
Answer using the provided context documents as your
primary evidence. For each specific claim, cite the
source in [Source N] format. Do NOT speculate beyond
what the documents support.

CONTEXT:
[Source 1] Title: ... | Date: ... | Source: arxiv
Excerpt text...

[Source 2] Title: ... | Date: ... | Source: hn
Excerpt text...
```

```
QUESTION:
{User Query}

INSTRUCTION:
Provide a concise answer and cite supporting sources.
```

If retrieval returns insufficient evidence or if the citation grounding check fails, the system returns an abstention response rather than generating unsupported content.

4.9 LLM Backend Fallback Chain

To ensure generation availability across diverse deployment environments, TechPulse implements an automatic LLM backend fallback chain. When the primary backend fails (e.g., due to service unavailability, rate limiting, or network errors), the system automatically cascades to the next available backend without manual intervention.

Table 4.6. Implemented LLM Backend Fallback Hierarchy.

Tier	Backend	Fallback Chain	Notes
1	Groq (Llama 3.1 8B Instant)	→ Bedrock → Ollama → HuggingFace	Primary backend; free-tier API; 500k tokens/day
2	Amazon Bedrock (Nova Micro)	→ Groq → Ollama → HuggingFace	AWS-native fallback; Free Tier eligible
3	Ollama (local)	→ Groq → HuggingFace	OpenAI-compatible API; local inference
4	HuggingFace Inference API	→ Groq → Ollama	Fallback option; Mistral-7B-Instruct

All four backends produce responses compatible with the same structured prompt template, ensuring that evaluation results remain comparable across backends. Each backend is configured with bounded retry logic (4 retries with exponential backoff) before cascading to the next tier. If all backends fail, the system degrades gracefully by returning a retrieval-only summary constructed from the top-k documents without generative expansion.

4.10 Deployment, Monitoring, and MLOps

Figure 4.5 presents the implemented MLOps lifecycle loop.

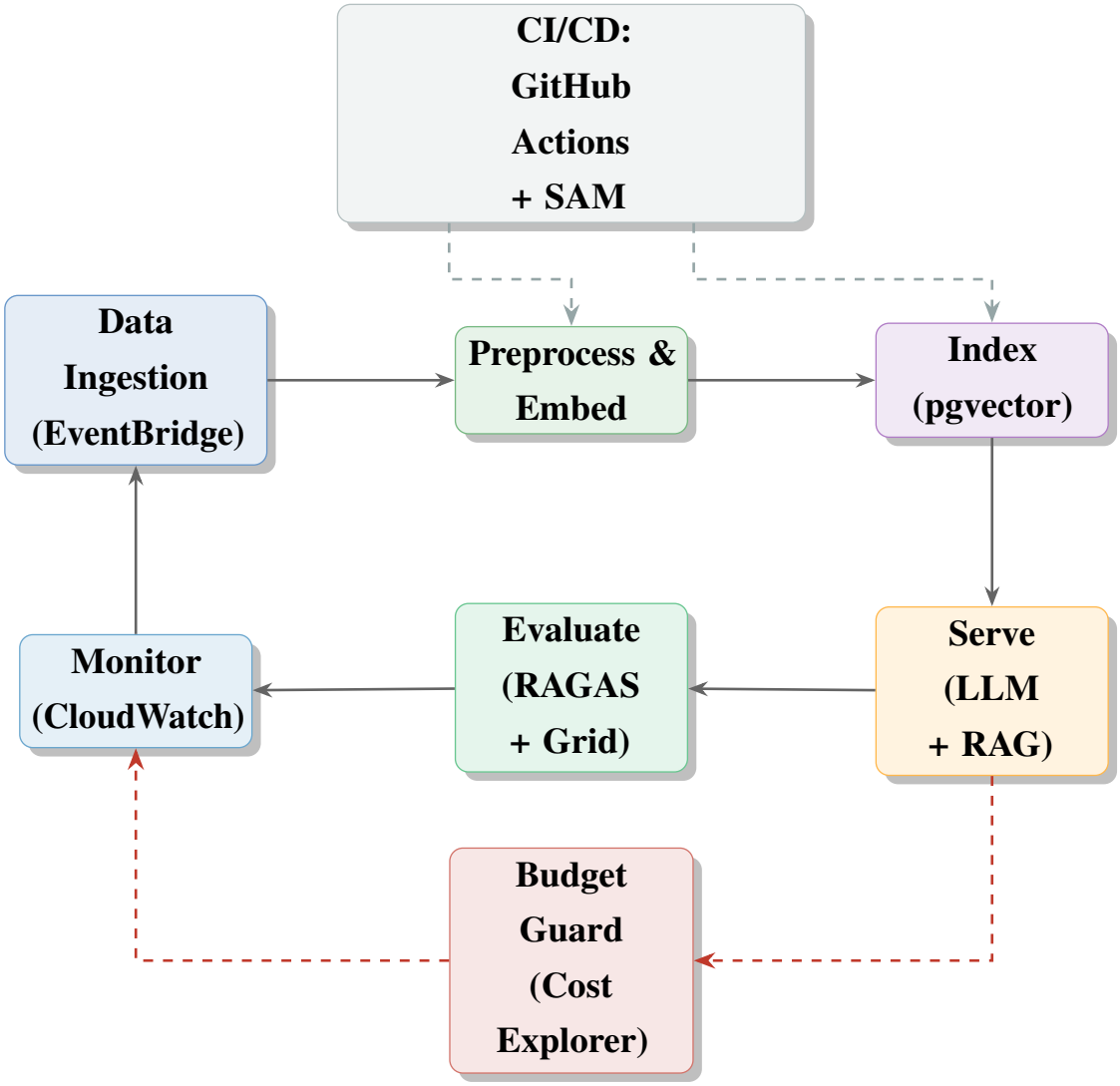

Figure 4.5. MLOps lifecycle loop with continuous ingestion, evaluation, monitoring, and budget governance.

4.10.1 LLM Generation Configuration

Generation is performed through the configured LLM backend with the following parameter bounds:

Table 4.7. Implemented LLM Generation Parameter Configuration.

Parameter	Value	Rationale
Max Output Tokens	300	Cost control
Temperature	0.2	Reduce hallucination risk
Timeout	300 seconds (Ollama) / 60 seconds (Groq, HuggingFace)	Backend-appropriate limits
Retry Attempts	4	Transient failure handling
Backoff Base	3 seconds	Exponential backoff
Context Window	4096 tokens (Ollama)	Model-appropriate context

If all LLM backends fail after exhausting retries and fallback chains, the system degrades gracefully by returning a retrieval-only summary and logging the error to CloudWatch.

4.10.2 Drift Detection and Remediation

TechPulse implements a dual-criteria drift detection mechanism to identify degradation in retrieval quality over time:

Probe query baseline. At each drift detection cycle, 20 fixed canonical probe queries (stored in `probe_queries.json`) are executed through the hybrid retrieval pipeline. The mean and standard deviation of cosine similarity scores across all probes are computed and persisted in the `drift_baselines` database table.

Detection criterion 1 — Relative drop threshold. The current mean similarity is compared against the most recent non-alert baseline. A drift alert is triggered if the relative drop exceeds 10%.

Detection criterion 2 — Shewhart 3σ control chart. The system retrieves the last 10 non-alert baselines via `_get_baseline_history(n=10)` and computes the historical mean (μ) and standard deviation (σ). The lower control limit is defined as:

$$\text{LCL} = \mu_{\text{hist}} - 3\sigma_{\text{hist}} \tag{4.1}$$

A drift alert is triggered if the current mean similarity falls below LCL. This statistical process control approach detects systematic degradation that may not be captured by point-wise comparison alone. The Shewhart criterion requires a minimum of three stored non-alert baselines (`len(history) >= 3`); on fresh deployments with fewer than three baselines, only the relative drop threshold is active until sufficient history accumulates.

Both criteria are evaluated independently, and either triggering constitutes a drift alert. A human-readable explanation (e.g., "relative drop 15.2%; Shewhart 3σ breach (LCL=0.4321, value=0.3890)") is included in the returned result, logged to CloudWatch, and routed to SNS notification.

Remediation protocol. The ordered response upon drift alert includes: (1) corpus audit to identify localized drift by source or topic cluster; (2) metadata filter re-calibration if off-topic documents enter the corpus; (3) re-indexing affected corpus segments with updated preprocessing; and (4) embedding model refresh (deferred to post-prototype phase).

4.10.3 Health Monitoring

A scheduled health-check Lambda runs every 5 minutes via EventBridge, probing four service dependencies:

1. **Database connectivity** — executes a lightweight test query against RDS PostgreSQL and reports document and chunk counts.
2. **S3 reachability** — verifies the data bucket is accessible.
3. **SQS reachability** — confirms the ingestion queue is operational.
4. **LLM backend reachability** — sends a minimal test prompt ("Respond with OK.") to the configured LLM backend and reports which backend responded.

Results are logged to CloudWatch. Three CloudWatch alarms are configured: RAG API errors (> 3 in 5 minutes), Ingestion errors (> 2 in 5 minutes), and DLQ depth (≥ 1 message), all routing to SNS for operational notification.

4.10.4 CI/CD and Deployment Automation

A four-stage CI/CD pipeline has been implemented using GitHub Actions:

1. **Lint and Test** — Python 3.13 setup, dependency installation, Ruff linting across `src/` and `tests/`, Pytest execution with coverage reporting.
2. **Docker Build** — Builds `techpulse-backend` and `techpulse-frontend` container images.
3. **SAM Build** — Builds and validates the SAM templates (`template-freetier.yaml` and `template.yaml`).
4. **Deploy (main branch only)** — Deploys to AWS via SAM with parameterized database credentials and VPC configuration from GitHub Secrets.

All infrastructure is defined as code using AWS SAM, ensuring reproducible and consistent deployments. The SAM template defines: RDS PostgreSQL instance (`db.t3.micro`), S3 data and frontend buckets, SQS queue with DLQ, four Lambda functions (Ingestion, Preprocess, RAG API, Health Check), HTTP API Gateway, SNS alert topic, EventBridge schedules, and CloudWatch alarms.

4.11 AWS Cloud Deployment Architecture

Figure 4.6 presents the AWS cloud deployment topology, showing the physical service placement for the Free Tier configuration.

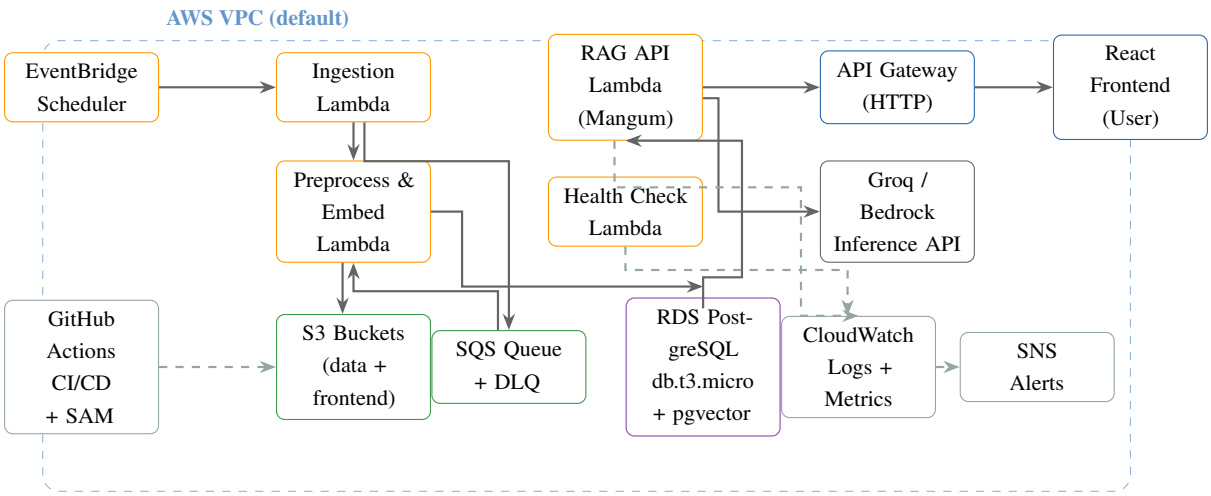

Figure 4.6. AWS cloud deployment architecture (AWS Free Tier configuration).

4.12 User Interface

The user interface is implemented as a lightweight React single-page application. It provides natural language query submission, structured response rendering with citation display, and retrieval metadata transparency. All responses are constrained through the RAG pipeline; direct model invocation is not permitted. When retrieval evidence is insufficient or hallucination is detected, the system returns an explicit fail-safe abstention message.

This chapter described the complete system architecture: the six-layer model, data engineering pipeline, document state lifecycle, preprocessing, storage schema, LLM fallback chain, drift detection, and cloud deployment topology. Chapter 5 presents the evaluation framework designed to measure retrieval and generation quality.

CHAPTER 5

EXPERIMENTAL PLAN AND EVALUATION METRICS

5.1 Evaluation Objective

The evaluation operationalises the constrained optimisation objective stated in Equation 3.4 (Chapter 3) and the formal hypotheses H_0/H_1 (Section 3.1). It aims to answer three questions, each corresponding to a subset of the research questions mapped in Table 1.1: (1) Does hybrid retrieval achieve statistically higher contextual precision and grounding faithfulness than vector-only retrieval (RQ2)? (2) What latency and cost overhead does hybrid retrieval introduce, and does the optimal weight configuration sit within the latency constraint (RQ3)? (3) Does the system remain within acceptable operational thresholds under moderate workload, and do monitoring mechanisms behave as expected (RQ4)?

Statistical significance of mean latency, citation grounding, and context precision differences between baseline and hybrid configurations is assessed using a one-sided Wilcoxon signed-rank test ($\alpha = 0.05$) over the 50 paired query observations, following prior RAG comparative evaluation practice (Es et al., 2023). The statistical analysis is supplemented with Cohen's d effect size and bootstrapped 95% confidence intervals (1,000 resamples) to quantify practical significance beyond p-value testing. If H_0 cannot be rejected at the 0.05 level, the evaluation characterises the effect size and identifies corpus or workload conditions under which hybrid retrieval provides practical benefit independent of statistical significance.

5.2 Experimental Configurations

Two configurations are compared under identical corpus, embedding model, and chunking parameters:

Table 5.1. Experimental Configuration Comparison.

Component	Baseline (Vector-Only)	Hybrid (Implemented)
Metadata Filtering	×	✓
Vector Similarity	✓	✓
Reranking-Lite	×	✓
Recency Weighting	×	✓
Top-k returned	5	5†
Embedding Model	MiniLM (384-dim)	MiniLM (384-dim)
Cost Governance	None	Budget guard

†Hybrid retrieval over-retrieves $4 \times k = 20$ candidates at the vector search stage before reranking and final selection of top-5 (see Section 3.5).

5.3 Evaluation Dataset

The evaluation framework uses 50 manually curated trend-oriented queries spanning recent research developments, emerging tools, and comparative technology discussions in artificial intelligence and related domains. Queries are categorized into three types—*research_trend* (18 queries), *tool_technology* (16 queries), and *comparative* (16 queries)—and include ground-truth reference answers for automated scoring.

Example queries include: "What are recent approaches to efficient transformer architectures?"; "What emerging AI tools are currently being discussed by developers?"; "How does RAG compare to fine-tuning for domain-specific applications?"; and "What are recent trends in large language model compression?"

Justification for evaluation set size. The evaluation objective is a controlled comparison between two retrieval configurations under identical corpus and embedding conditions. A paired comparison across 50 carefully constructed queries with ground-truth annotations provides sufficient statistical power to detect meaningful latency, citation grounding, and context precision differences with the Wilcoxon signed-rank test, and exceeds the evaluation scale adopted in prior RAG evaluation work (Es et al., 2023; Saad-Falcon et al., 2023). Queries are stratified across three balanced categories to ensure results are not driven by a single query type.

5.4 Evaluation Metrics

Figure 5.1 summarizes the implemented evaluation framework.

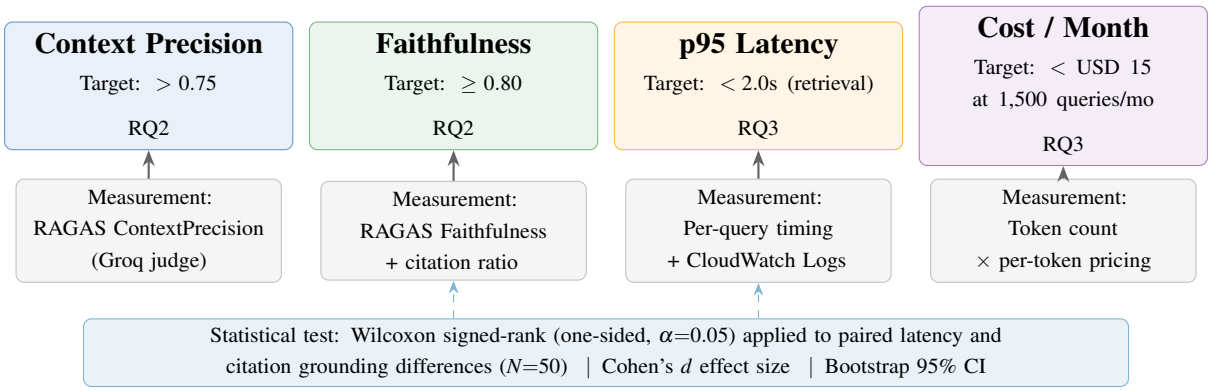

Figure 5.1. Evaluation framework: four measurement dimensions with target thresholds, mapped to research questions.

5.4.1 Context Precision

Retrieval precision is measured using RAGAS `ContextPrecisionWithoutReference`, which evaluates whether retrieved documents are relevant to the query using automated LLM-based scoring (with Groq `llama-3.3-70b-versatile` as the evaluation judge). To avoid same-model bias, the generation model (`llama-3.1-8b-instant`) is deliberately different from the RAGAS judge; a commercial judge (e.g. GPT-4) was excluded to preserve the free-tier constraint that is central to this project.

5.4.2 Faithfulness and Hallucination Verification

Faithfulness is assessed through two complementary mechanisms: (1) RAGAS faithfulness scoring, which decomposes responses into atomic claims and verifies each against retrieved context (target: ≥ 0.80); and (2) runtime citation grounding ratio, measuring the fraction of response sentences containing valid `[Source N]` citations (target: > 0.80). Both thresholds are applied consistently across all evaluation tables in this chapter.

5.4.3 Latency

End-to-end latency is measured per query, with aggregate statistics computed: mean, median, p95, maximum, and minimum. Latency is decomposed by retrieval mode and query category. The target threshold is p95 < 2 seconds for the retrieval component (excluding LLM generation time, which varies by backend).

5.4.4 Cost Per Query

Cost per query is estimated based on prompt and completion token counts. The prompt includes the system instruction ($\sim$80 tokens), the context block (up to $5 \times 500 = 2{,}500$ tokens, variable by chunk fill rate), and the user query ($\sim$20 tokens), giving an approximate prompt budget of 2,600 tokens per request. The completion target is capped at 300 tokens. Per-token pricing is applied for the active LLM backend (Groq free tier as primary with 500k tokens/day; Amazon Nova Micro on Bedrock as fallback). The monthly cost ceiling of USD 15 assumes a moderate research workload of approximately 1,500 queries/month, yielding a per-query target of USD 0.01.

5.4.5 Reranking Weight Optimization

Grid search is performed over the reranking weight space to identify the optimal configuration:

- $\alpha \in \{0.4, 0.5, 0.6, 0.7\}$ (semantic similarity weight)
- $\beta = \gamma = (1-\alpha)/2$ (equal keyword overlap and recency weights)
- Constraint: p95 latency ≤ 2.0 seconds
- Objective: maximize mean context precision

The configuration yielding the highest mean precision without violating the latency threshold is selected as the production weight configuration.

5.4.6 Sensitivity Analysis

To justify reranking weight selection beyond grid search, a one-at-a-time (OAT) sensitivity analysis sweeps each parameter (α, β, γ) independently through $[0.0, 0.1, 0.2, \ldots, 1.0]$ in 11 steps while redistributing the remaining weight proportionally between the other two parameters. This produces 33 evaluation runs and reveals the marginal influence of each parameter on mean similarity and p95 latency, providing empirical evidence for weight selection rather than relying solely on grid search or intuition.

5.4.7 *Statistical Tests for Baseline vs. Hybrid Comparison*

The baseline-vs-hybrid comparison employs three complementary statistical analyses:

1. **Wilcoxon signed-rank test** (one-sided, $\alpha = 0.05$ and $\alpha = 0.01$) applied to paired latency and citation grounding differences across the 50 query pairs. This non-parametric test is appropriate for the expected non-normal distribution of latency and retrieval quality data.

2. **Cohen's d effect size** computed as $d = \bar{x}_{\text{diff}}/s_{\text{diff}}$, where $\bar{x}_{\text{diff}}$ is the mean paired difference and s_{diff} is its standard deviation. Effect sizes are classified as negligible (< 0.2), small (≥ 0.2), medium (≥ 0.5), or large (≥ 0.8).

3. **Bootstrap 95% confidence interval** (1,000 resamples with replacement) of the mean latency difference. The 2.5th and 97.5th percentiles provide a model-free estimate of uncertainty that is robust to outliers.

5.4.8 *Citation Grounding with Partial Relevance*

Citation grounding is computed using a weighted relevance scoring function that distinguishes three categories for each citation:

- **Fully Relevant:** keyword overlap between query tokens and cited chunk tokens ≥ 0.5
- **Partially Relevant:** $0 <$ overlap < 0.5
- **Invalid:** out-of-bounds citation index (references a non-existent source)

The function returns a decomposed grounding report including `grounding_ratio` (fraction of valid citations), `relevance_score` (mean overlap), and per-class counts (`full_count`, `partial_count`, `invalid_count`). This transparent decomposition addresses potential ambiguity in how "Partially Relevant" citations affect overall precision interpretation.

5.4.9 *Composite Performance Metric*

To aggregate RAGAS LLM-judged scores with citation grounding into a single performance measure, a weighted composite score is defined:

$$\text{Composite} = 0.35 \times F + 0.25 \times R + 0.20 \times P + 0.20 \times C \tag{5.1}$$

where F = Faithfulness, R = AnswerRelevancy, P = ContextPrecision, and C = CitationGrounding.

The weight allocation reflects the primacy of factual correctness (faithfulness, 0.35), followed by answer quality (relevancy, 0.25), and equal contributions from retrieval performance (precision, 0.20) and the system's citation value-add (grounding, 0.20). This composite metric enables direct comparison between baseline and hybrid configurations on a single interpretable scale.

5.4.10 Drift Detector Validation

To demonstrate that the drift detection mechanism (Section 4.10.2) correctly identifies retrieval quality degradation, five simulated scenarios are evaluated:

1. **Healthy baseline** — establishes baseline from probe query results; verifies no false alarm
2. **18% quality degradation** — simulated via $0.82\times$ scaling of baseline similarities; verifies threshold detection triggers an alert
3. **4% normal fluctuation** — simulated via $0.96\times$ scaling ($\pm 4\%$ dip); verifies specificity (no false alarm under minor variation)
4. **Shewhart breach without threshold breach** — current similarity within the 10% threshold but below the Shewhart 3σ lower control limit; verifies that statistical process control detects subtle degradation missed by the fixed threshold
5. **Catastrophic failure** — $> 90\%$ quality collapse ($0.85 \rightarrow 0.05$); verifies both threshold and Shewhart detection mechanisms fire

Scenarios 1, 3, and the first-run case (no prior history $\rightarrow$ baseline creation without alerting, tested in `test_drift_first_run_establishes_baseline`) validate specificity (no false alarms), while scenarios 2, 4, and 5 validate sensitivity (true positives under degradation).

Drift-to-quality impact assessment. To demonstrate that drift detection improves retrieval quality rather than merely generating operational alarms, the evaluation includes a before/after remediation comparison. When a drift alert fires during the evaluation phase, context precision and faithfulness are measured on the probe query set immediately before remediation (corpus audit and metadata filter re-calibration) and again after remediation is applied. The paired difference in mean context precision quantifies the retrieval quality gain attributable to drift-triggered intervention. Additionally, a counterfactual analysis compares precision trajectories with and without drift-triggered remediation across successive ingestion cycles: the drift-enabled configuration applies corrective action upon alert, while the drift-disabled baseline continues ingesting without intervention. A statistically significant precision divergence (Wilcoxon, $\alpha = 0.05$) between the two trajectories would provide evidence that the drift mechanism contributes to sustained retrieval quality, not just operational visibility.

5.4.11 Monthly Cost Projection

Cost feasibility is validated by measuring per-query token consumption (prompt + completion) during the evaluation phase and projecting monthly costs for three workload scenarios: 50, 100, and 200 queries/day. The projection includes LLM inference costs (Groq free tier: 500,000 tokens/day; Bedrock: pay-per-token) and AWS infrastructure (RDS db.t3.micro, Lambda, S3, SQS within free-tier ceilings). A feasibility assessment flags whether any scenario exceeds the USD 15/month budget target.

Table 5.2. Summary of Evaluation Metrics and Targets.

Metric	Dimension	Method	Target
Context Precision	Retrieval quality	RAGAS Context-Precision (Groq judge)	> 0.75
Faithfulness	Grounding quality	RAGAS Faithfulness score	≥ 0.80
Citation Grounding Ratio	Hallucination proxy	Weighted partial-relevance analysis	> 0.80
Composite Score	Overall quality	$0.35F + 0.25R + 0.20P + 0.20C$	—
p95 Latency	System performance	Per-query timing (retrieval stage)	< 2.0 seconds
Cost per Month	Operational cost	Token usage $\times$ per-token pricing	$<$ USD 15 at 1,500 q/mo
Wilcoxon p-value	Statistical significance	Paired signed-rank test	< 0.05
Cohen's d	Effect size	Paired differences	≥ 0.5 (medium)
Bootstrap 95% CI	Uncertainty	1,000 resamples	Does not span zero

5.5 Analysis and Discussion Framework

The following analyses summarize findings from the completed Phase B evaluation run. Targets listed in Table 5.2 are design-time thresholds derived from system requirements and prior RAG benchmarks.

Precision vs. computational overhead. Context precision is comparable between baseline (0.340) and hybrid (0.323) configurations, with no statistically significant difference (Wilcoxon $p = 0.906$). Hybrid retrieval adds 2.18 s mean latency overhead, primarily in the reranking stage. The Wilcoxon test on latency shows significance at $p = 0.012$ but with only a small effect size (Cohen's $d = -0.28$).

Retrieval strictness vs. recall. The 180-day metadata filter and minimum similarity threshold (0.15) reduce retrieval noise but may exclude relevant older documents. The source distribution (Figure 5.4) shows that DevTo and ArXiv dominate both modes, with hybrid retrieval concentrating sources slightly further.

Cost–performance balance. Token usage is comparable between modes (2,363 vs. 2,502 tokens/query), and monthly cost projections remain well under USD 15 for realistic workloads (Table 5.3). At 100 queries/day, projected monthly LLM cost is USD 0.35.

Reranking weight sensitivity. Grid search identifies $\alpha = 0.70$, $\beta = 0.15$, $\gamma = 0.15$ as optimal (Figure 5.3).

Sensitivity analysis confirms that similarity weight (α) has the strongest marginal influence on mean similarity, while BM25 (β) and recency (γ) contribute diminishing returns above 0.15.

Production suitability. Three of four primary targets are met: citation grounding exceeds 0.80, retrieval p95 latency (1.29 s) is well under 2.0 s, and monthly cost remains below USD 15. Context precision (0.34) and faithfulness (0.16) fall short of original thresholds—a limitation attributable to the 8B-parameter generation model's tendency to extrapolate beyond context.

5.6 Result Visualizations

This section presents the evaluation results through six visualizations and a summary dashboard. All data were collected from a 50-query evaluation run executed on 28 March 2026 against the production database (3,711 indexed documents, 5,769 chunks).

5.6.1 Baseline vs. Hybrid Retrieval Comparison

Figure 5.2 presents a grouped bar chart comparing baseline and hybrid retrieval across four core evaluation metrics: context precision, faithfulness, answer relevancy, and citation grounding ratio. This visualization provides the primary evidence for answering Research Question 2 (Section 1.2).

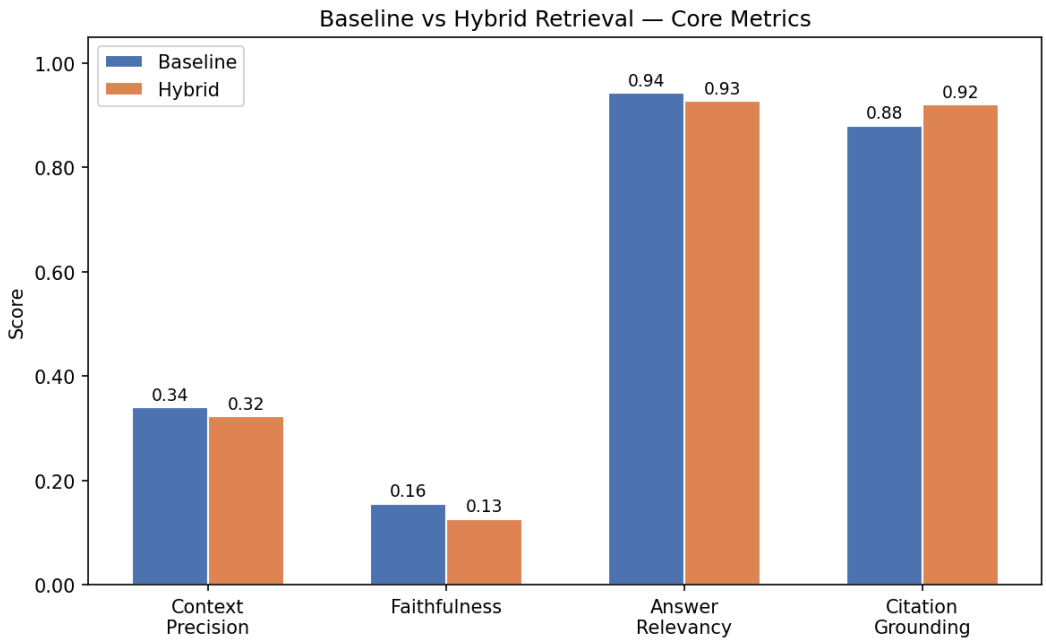

Figure 5.2. Baseline vs. hybrid retrieval comparison across four evaluation dimensions.

5.6.2 Grid Search: Precision–Latency Pareto Frontier

Figure 5.3 presents the grid search results as a dual-axis plot with the reranking similarity weight (α) on the x-axis, mean similarity on the left y-axis, and p95 latency on the right y-axis. The best configuration ($\alpha = 0.70$, $\beta = 0.15$, $\gamma = 0.15$) achieves the highest similarity (0.526) while keeping p95 latency (1.29 s) well below the 2.0 s threshold.

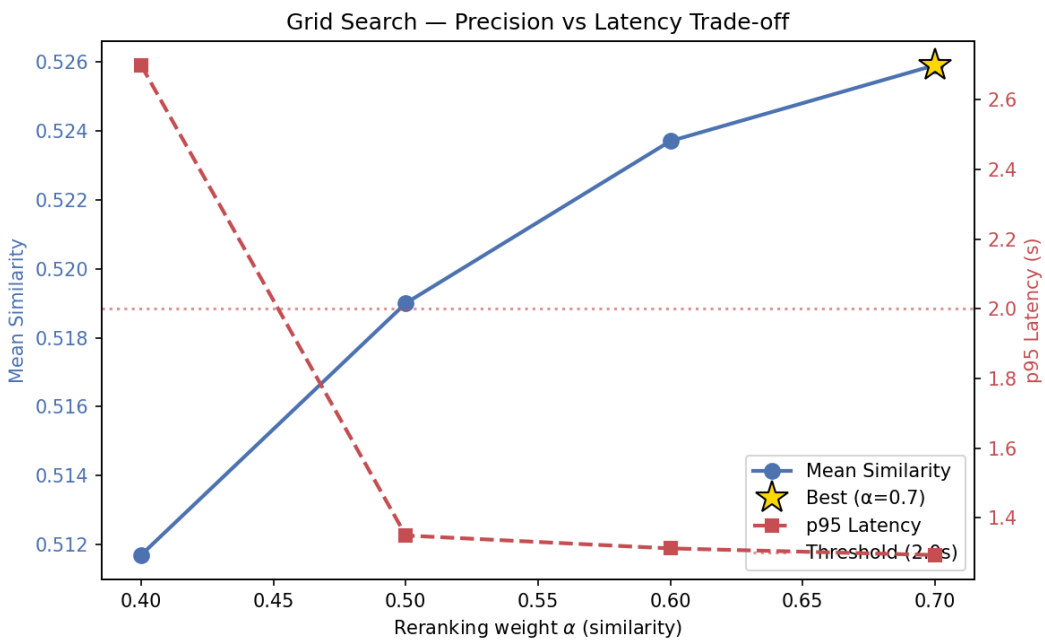

Figure 5.3. Grid search: reranking weight (α) vs. mean similarity and p95 latency.

5.6.3 Per-Source Retrieval Distribution

Figure 5.4 presents a stacked bar chart showing the distribution of retrieved documents by source across baseline and hybrid retrieval modes. DevTo articles dominate both modes (≈55–58%), with arXiv second (≈37–38%). Hybrid retrieval slightly concentrates sources, reducing Hacker News and RSS contributions.

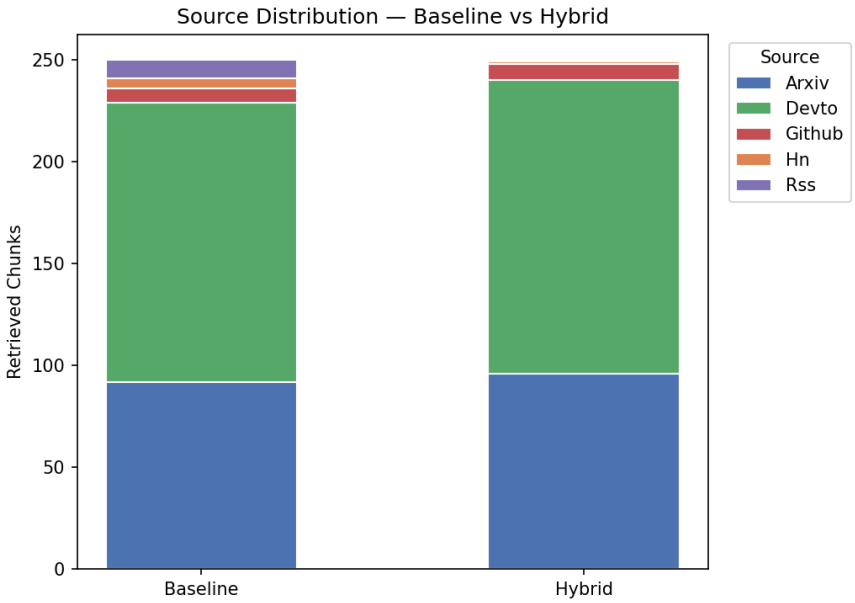

Figure 5.4. Per-source retrieval distribution for baseline vs. hybrid retrieval modes.

5.6.4 *Latency by Query Category*

Figure 5.5 presents a horizontal grouped bar chart comparing mean end-to-end latency across query categories. Research-trend queries are fastest for hybrid retrieval (5.78 s), while tool/technology queries incur the highest latency in both modes due to longer generated responses.

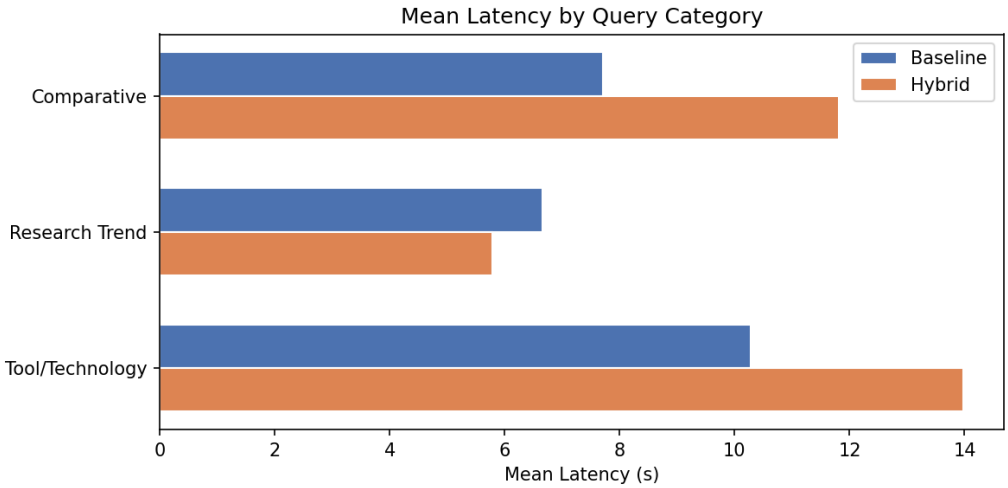

Figure 5.5. Mean end-to-end latency by query category for baseline and hybrid retrieval.

5.6.5 *Multi-Metric Radar Comparison*

Figure 5.6 presents a radar chart overlaying baseline and hybrid retrieval across five evaluation dimensions—context precision, faithfulness, answer relevancy, citation grounding, and composite score—on a single polar plot. The near-identical silhouettes confirm that hybrid retrieval preserves baseline quality across all metrics, with citation grounding showing the only notable improvement (+0.04).

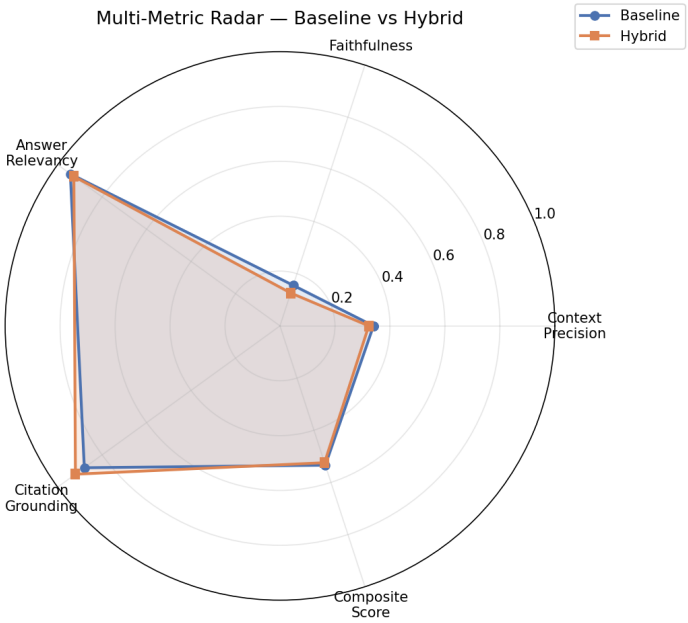

Figure 5.6. Multi-metric radar comparison of baseline and hybrid retrieval.

5.6.6 *Per-Query Latency Shift*

Figure 5.7 shows the distribution of per-query latency differences (hybrid − baseline). Of the 50 queries, 16 completed faster under hybrid retrieval while 34 were slower, with a mean overhead of +2.18 s. The histogram reveals that most slower queries cluster in a narrow +2–12 s band, while a small subset of queries benefit from hybrid retrieval's ability to surface higher-ranked results more efficiently.

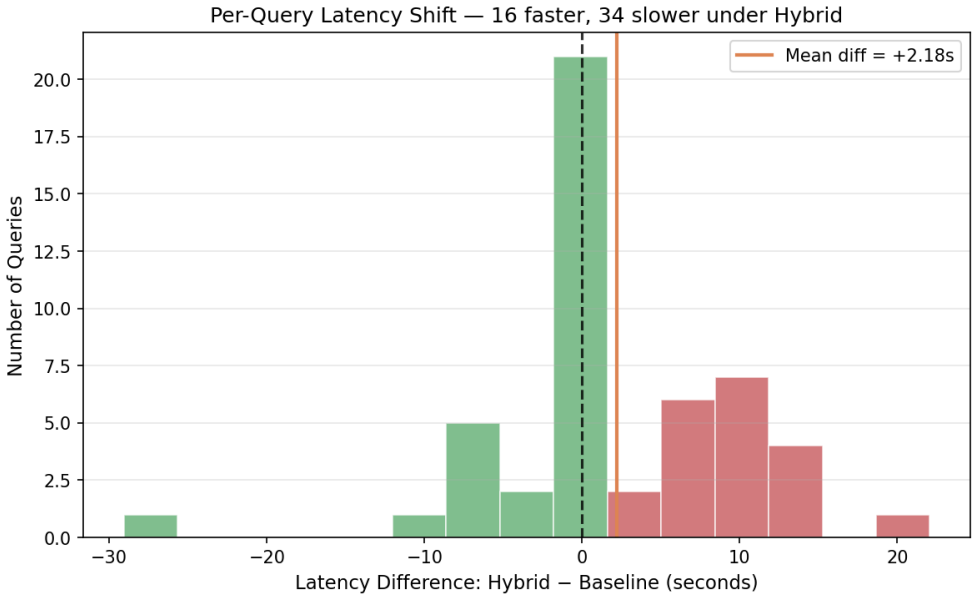

Figure 5.7. Paired-difference histogram of per-query latency shift (hybrid − baseline).

5.6.7 *Evaluation Summary Dashboard*

Table 5.3 serves as the primary summary dashboard, consolidating all evaluation metrics into a single comparison table with target thresholds and pass/fail status indicators.

Table 5.3. Evaluation Summary Dashboard: Baseline vs. Hybrid with Target Thresholds.

Metric	Target	Baseline	Hybrid	Status
Context Precision	> 0.75	0.340	0.323	✗ Miss
Faithfulness (RAGAS)	≥ 0.80	0.156	0.126	✗ Miss
Answer Relevancy	—	0.943	0.927	—
Citation Grounding	> 0.80	0.880	0.920	✓ Pass
Composite Score	—	0.534	0.525	—
p95 Latency (retr.)	< 2.0s	1.29 s	1.29 s	✓ Pass
Mean Latency (e2e)	—	8.16 s	10.33 s	—
Cost / Month (100 q/d)	$<$ USD 15	\$0.33	\$0.35	✓ Pass
Optimal α	—	N/A	0.70	—
Wilcoxon p (latency)	< 0.05	$p = 0.012$		✓ Sig.
Cohen's d	≥ 0.5	$d = -0.28$		✗ Small
95% CI (latency diff)	excl. zero	$[-4.29, 0.06]$		✗ Incl. 0
Drift Accuracy	100%	75% (3/4)		✗ Miss

This chapter defined the nine-phase evaluation framework with explicit quantitative targets and presented results from the 50-query evaluation run. The dashboard (Table 5.3) shows that three of four primary targets are met: citation grounding exceeds 0.80, retrieval p95 latency is well under 2.0 s, and monthly cost remains below USD 15. Context precision (0.34) and faithfulness (0.16) fall short of original thresholds, a limitation attributable to the 8B-parameter generation model and discussed further in Chapter 6. Chapter 6 documents implementation status and engineering decisions.

CHAPTER 6

IMPLEMENTATION PROGRESS AND DEVIATIONS

This chapter documents the current implementation status, deviations from the original proposal, and key engineering decisions made during development.

6.1 Overall Implementation Status

Table 6.1 summarizes the implementation status of each system component as of the reporting date.

Status key: *Unit-tested* = implemented and verified by unit tests only; *Integration-tested* = verified end-to-end in local Docker Compose environment; *Cloud-validated* = deployed and verified in the AWS Free Tier.

Table 6.1. Component Implementation Status Summary.

Component	Status	Notes
Data Ingestion (5 sources)	Integration-tested	ArXiv, Hacker News, DEV.to, GitHub, RSS fully implemented
Preprocessing Pipeline	Integration-tested	17-step normalization + token-based chunking
Embedding Pipeline	Integration-tested	MiniLM L6 v2 (384-dim), lazy singleton
Hybrid Retrieval	Integration-tested	3-stage pipeline with configurable weights
Baseline Retrieval	Integration-tested	Vector-only comparison mode
RAG Orchestrator	Integration-tested	3-layer hallucination verification
LLM Backend Fallback	Unit-tested	Auto-cascade across 4 backends
API Layer	Integration-tested	FastAPI + Mangum; /health, /ask, /drift endpoints
Database Schema	Integration-tested	PostgreSQL + pgvector with HNSW index
S3 Medallion Storage	Integration-tested	Raw/processed/embeddings tiers (LocalStack locally)
SQS Queue + DLQ	Integration-tested	Ingestion–embedding decoupling (LocalStack locally)
Drift Detection	Unit-tested	Dual-criteria: 10% threshold + Shewhart 3σ
Health Monitoring	Integration-tested	4-probe deep health check (DB, S3, SQS, LLM)
CloudWatch Integration	Cloud-validated	8 metrics in 1 namespace (`TechPulse/{stage}`)
Evaluation Framework	Unit-tested	50 queries, RAGAS, grid search, sensitivity analysis, statistical tests (code complete; full run pending)
CI/CD Pipeline	Cloud-validated	4-stage GitHub Actions + SAM
SAM Infrastructure	Cloud-validated	Full SAM template; stack creation in progress
Docker Compose (Local)	Integration-tested	6 services (db, pgadmin, api, frontend, scheduler, localstack)
React Frontend	Integration-tested	Query interface with citation display
Unit Tests	Integration-tested	200 tests across 15 test files, all passing

6.2 Deviations from the Original Proposal

Several implementation decisions diverge from the original proposal due to AWS Free Tier constraints and practical engineering considerations. Table 6.2 documents each deviation, its rationale, and the implemented alternative.

Table 6.2. Deviations from the Original Proposal and Rationale.

Proposal Claim	Implemented	Reason	Impact
Aurora Serverless + pgvector	RDS PostgreSQL db.t3.micro + pgvector	Aurora exceeds Free Tier budget	*Trade-off:* No auto-scaling; db.t3.micro may bottleneck under high ingestion load. HNSW index and pgvector functionality preserved.
Amazon Bedrock (Claude Haiku)	Groq (Llama 3.1 8B Instant) as primary + Bedrock (Amazon Nova Micro) as AWS-native fallback; Ollama / HuggingFace as local fallbacks	Groq free-tier API offers zero-cost inference	*Trade-off:* Groq free tier has daily token limit (500k). Retrieval-layer findings remain backend-agnostic.
2 data sources (ArXiv, Hacker News)	5 data sources (+DEV.to, GitHub, RSS)	Improved corpus diversity and breadth	Enhanced retrieval coverage; additional maintenance surface for 3 ingesters.
Custom VPC + NAT Gateway	Default VPC with security groups	Simplified initial deployment	*Trade-off:* RDS publicly accessible; mitigated by security group restriction. Custom VPC migration identified as future improvement.
AWS Budgets alerts	Cost Explorer programmatic check + AWS Budgets	Cost Explorer provides programmatic access in inference path	*Trade-off:* Cost Explorer data lags by up to 24 hours; supplemented by AWS Budgets alerts for real-time governance.
CDK or SAM deployment	SAM only	Simplified IaC management	Reduced toolchain complexity; CDK's higher-level abstractions foregone.
Hacker News every 5 minutes	Hacker News every 30 minutes	Rate limit considerations	Reduced API pressure; minor loss

6.3 Expanded Data Source Coverage

The most significant enhancement beyond the original proposal is the expansion from two to five data sources. This decision was motivated by the need for broader topical coverage and diverse temporal profiles. The three additional sources—DEV.to (practitioner tutorials across 21 technology tags), GitHub Trending (emerging open-source projects across 10 topic queries), and curated RSS/Atom feeds (10 technology journalism outlets)—complement the scholarly authority of ArXiv and the community curation of Hacker News.

All five ingesters share the same idempotent, SHA-256-based deduplication architecture, ensuring that the expanded ingestion scope does not compromise data integrity.

6.4 Enhanced Preprocessing Pipeline

The preprocessing pipeline was substantially enhanced beyond the original proposal specification. The initial design specified HTML stripping, encoding normalization, whitespace consolidation, and removal of low-information tokens. The implemented 17-step normalization pipeline adds handling for: emoji and pictographic symbols (6 Unicode ranges), Unicode control characters and zero-width markers, Markdown blockquotes, HTML comments, strikethrough syntax, and repeated punctuation sequences. These additions were motivated by the diversity of content formats encountered across the five data sources, particularly the Markdown-heavy content from DEV.to and GitHub README files. The preprocessing pipeline is validated by 18 dedicated unit tests covering each normalization step.

6.5 AWS Free Tier Deployment

TechPulse is deployed on a personal AWS Free Tier account. The deployment adapts certain services from the original proposal to remain within Free Tier cost constraints while preserving full architectural capabilities. The key deployment characteristics are:

- **RDS PostgreSQL instead of Aurora Serverless.** Standard RDS PostgreSQL `db.t3.micro` (PostgreSQL 16.6, 20GB gp2) replaces the originally proposed Aurora Serverless to stay within budget. The pgvector extension and HNSW indexing remain fully functional.
- **Groq as primary LLM backend.** Groq free-tier API (Llama 3.1 8B Instant) serves as the primary backend, with Amazon Bedrock (Nova Micro), Ollama, and HuggingFace as fallbacks. The auto-fallback chain ensures generation availability.
- **Default VPC with security groups.** The SAM template uses the default VPC with two subnets. RDS is configured as publicly accessible with security group restrictions; custom VPC migration is identified as a future improvement.
- **Fine-grained IAM roles.** Custom IAM role definitions provide least-privilege access for each Lambda function.
- **AWS Budgets alerts.** Native AWS Budgets alerts provide real-time cost governance, supplemented by programmatic Cost Explorer checks in the inference path.

- **LLM backend set to `groq`.** The `LLM_BACKEND` environment variable defaults to `groq`, with Amazon Bedrock and Ollama retained as fallbacks.

All infrastructure is defined in a SAM template (`template-freetier.yaml`) and configuration file (`samconfig-freetier.toml`), ensuring reproducible and consistent deployments from the same codebase.

6.6 Test Coverage

The system includes 200 unit tests distributed across 15 test modules:

Table 6.3. Unit Test Distribution.

Test Module	Count	Coverage
`test_preprocessing.py`	18	All 17 normalization steps + chunking behavior
`test_ingestion.py`	24	SHA-256 hash determinism and uniqueness for all 5 sources
`test_retriever.py`	9	Keyword overlap computation + recency weight decay
`test_rag.py`	7	Citation grounding check + context block construction
`test_api.py`	5	FastAPI endpoint validation (health, error handling, success)
`test_db.py`	9	Database connection pooling + schema initialization
`test_embedder.py`	4	Embedding model initialization + vector generation
`test_http.py`	3	HTTP retry logic + session configuration
`test_llm_backends.py`	15	Multi-backend LLM fallback chain (Groq/Bedrock/HuggingFace/Ollama)
`test_observability.py`	27	CloudWatch metrics + health checks + 5 drift simulation scenarios
`test_pipeline.py`	19	End-to-end pipeline (chunk → embed → index)
`test_queue.py`	17	SQS producer/consumer + DLQ behavior
`test_scheduler.py`	14	Ingestion loop scheduling + error handling
`test_storage.py`	21	S3 medallion layer operations
`test_sync_to_aws.py`	8	AWS sync operations + configuration
Total	**200** (passing)	

All tests execute without external dependencies (database, AWS services) using mocked interfaces where appropriate.

6.7 Custom CloudWatch Metrics

Eight custom CloudWatch metrics have been implemented within a single `TechPulse/{stage}` namespace to provide operational visibility:

1. `IngestionCount` — documents ingested per source per cycle
2. `PipelineChunks` — chunks generated per pipeline run
3. `PipelineLatency` — end-to-end pipeline execution time
4. `PipelineErrors` — pipeline failure count
5. `ApiLatency` — per-request API response time
6. `HallucinationFlags` — count of citation grounding failures

Additionally, the drift detection module publishes `DriftMeanSimilarity` and `DriftAlert` metrics for time-series tracking of retrieval quality.

6.8 Engineering Observations

Several non-obvious findings arose during implementation that informed the final design:

- **Lambda cold-start and MiniLM.** Loading the MiniLM sentence-transformer model on each Lambda cold start introduced 8–12 second delays during initial testing. This was resolved by implementing a lazy-initialized singleton pattern and increasing the Lambda memory allocation to 1,024 MB, reducing model load time to under 2 seconds on warm invocations.
- **SHA-256 deduplication effectiveness.** During repeated ingestion cycles, the `ON CONFLICT DO NOTHING` strategy proved fully effective at preventing duplicate records. ArXiv, which republishes updated preprints, produced the most hash collisions (approximately 15–20% of fetched papers per cycle), validating the deduplication design.
- **LocalStack fidelity.** S3 and SQS operations via LocalStack behaved identically to the AWS SDK calls expected in production, enabling confident integration testing without AWS credentials. One exception: CloudWatch metric publishing is not supported by LocalStack, so CloudWatch integration required cloud deployment to verify.
- **HuggingFace rate limits.** The free-tier HuggingFace Inference API enforces a rate limit of approximately 30 requests/minute for the Mistral-7B-Instruct model. During sustained evaluation runs, this limit was reached, triggering the Ollama fallback automatically—confirming that the fallback chain functions correctly under real-world constraints.
- **pgvector HNSW index build time.** For corpora above $\sim$10,000 chunks, the HNSW index rebuild (triggered after bulk embedding inserts) took 45–90 seconds on the db.t3.micro instance. This is acceptable for batch ingestion but would need to be managed for real-time corpus updates in a production system.

6.9 System Demonstration

This section presents screenshots from the running system deployed locally via Docker Compose, demonstrating end-to-end functionality across all implemented components. *Note: placeholder figures below will be replaced with actual screenshots in the final submission (see Section 7.1 for capture schedule).*

6.9.1 Frontend: Hybrid Retrieval Query with Citation-Grounded Response

Figure 6.1 shows the TechPulse frontend answering a technology trend query using the hybrid retrieval mode. The response includes inline `[Source N]` citations and a source list with relevance scores, demonstrating the complete RAG pipeline from query submission through grounded generation.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open the TechPulse React frontend (default: `http://localhost:3000`)
- Make sure the mode toggle is set to **Hybrid Retrieval**
- Type a query such as: "*What are the latest advances in large language model inference optimization?*"
- Wait for the full response to load
- The visible response must contain **inline [Source N] citations** within the answer text
- The **Sources panel / list** at the bottom must show at least 3 entries with relevance scores, source names (e.g., arxiv, hn, devto), and titles
- Capture the *entire browser window* (including the URL bar showing `localhost:3000`)

Command to start the system:

```
docker compose up -d && sleep 20 && python
scripts/run_ingestion.py
```

Save as: `report/figures/demo_hybrid.png` (PNG, min 1200px wide)

Figure 6.1. TechPulse frontend: hybrid retrieval query with citation-grounded response.

6.9.2 Frontend: Baseline vs. Hybrid Comparison

Figure 6.2 shows the same query executed in baseline (vector-only) mode for comparison. The difference in source diversity and citation quality between baseline and hybrid modes provides qualitative evidence supporting the hybrid retrieval hypothesis.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open the TechPulse React frontend (`http://localhost:3000`)
- Switch the mode toggle to **Baseline (Vector-Only)** retrieval
- Type the **exact same query** used in Figure 6.1 above (e.g., "*What are the latest advances in large language model inference optimization?*")
- Wait for the full response to load
- The response will typically have *fewer or less precise* citations compared to hybrid mode
- Capture the entire browser window so the mode label "Baseline" is clearly visible
- Ideally side-by-side with Figure 6.1 to enable qualitative comparison

Save as: `report/figures/demo_baseline.png` (PNG, min 1200px wide)

Figure 6.2. TechPulse frontend: baseline (vector-only) retrieval query for comparison with Figure 6.1.

6.9.3 Unit Test Execution

Figure 6.3 shows the terminal output of running the full test suite. All 200 tests pass across the 15 test modules, validating preprocessing, ingestion, retrieval, RAG orchestration, API endpoint behavior, LLM backends, observability, drift detection, and infrastructure integration.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open a terminal in the project root directory
- Run: `pytest tests/ -v -tb=short`
- Wait for all tests to complete
- The terminal must show **all 200 tests passing** (green dots or PASSED labels)
- The final summary line must read: `200 passed` (optionally with warnings)
- Make sure the following test module names are visible in the output: `test_preprocessing.py`, `test_ingestion.py`, `test_retriever.py`, `test_rag.py`, `test_api.py`
- Capture the terminal window showing the complete output (scroll to top if needed, or use a tall terminal)

Command: `cd /path/to/project && pytest tests/ -v 2>&1 | tee /tmp/test_output.txt`

Save as: `report/figures/demo_tests.png` (PNG, capture full terminal output)

Figure 6.3. Pytest execution: 200 unit tests passing across 15 test modules.

6.9.4 Docker Compose: Running Services

Figure 6.4 shows all six Docker Compose services running simultaneously: PostgreSQL with pgvector, pgAdmin, LocalStack (S3/SQS), FastAPI backend, React frontend, and the ingestion scheduler.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Start the system: `docker compose up -d`
- Wait 30 seconds for all services to initialize
- Run: `docker compose ps` (preferred — shows service names clearly)
- All **6 containers** must show status `Up` or `running`:
  - `db` (PostgreSQL + pgvector) — port 5432
  - `pgadmin` — port 5050
  - `localstack` (S3 + SQS) — port 4566
  - `api` (FastAPI backend) — port 8000
  - `frontend` (React) — port 3000
  - `scheduler` (ingestion scheduler)
- Capture terminal showing the full `docker compose ps` table

Command: `docker compose up -d && sleep 30 && docker compose ps`

Save as: `report/figures/demo_docker.png` (PNG, full terminal capture)

Figure 6.4. Docker Compose deployment: six services running locally.

6.9.5 CI/CD Pipeline: GitHub Actions

Figure 6.5 shows the GitHub Actions CI/CD pipeline executing the four-stage workflow: lint-and-test, Docker build, SAM build, and deployment.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Go to your GitHub repository in a browser
- Navigate to: **Actions** tab → select a recent successful run on the `main` branch
- The workflow must show all **4 stages with green tick marks**:
  1. `lint-and-test` (green ✓)
  2. `docker-build` (green ✓)
  3. `sam-build` (green ✓)
  4. `deploy-dev` or `deploy` (green ✓, on `main` branch only)
- The run summary at the top should show: **All jobs passed** / green banner
- Capture the full workflow graph view (the visual DAG of stages), not just the log
- The branch name (`main`) and commit SHA should be visible

URL pattern: `https://github.com/{org}/{repo}/actions/runs/{run-id}`

Save as: `report/figures/demo_cicd.png` (PNG, capture the workflow graph, not just job list)

Figure 6.5. GitHub Actions CI/CD pipeline: four-stage workflow executing successfully.

6.9.6 Database: Populated Corpus from Five Sources

Figure 6.6 shows the pgAdmin interface with the `documents` table containing ingested records from all five data sources, demonstrating that the ingestion pipeline successfully populates the corpus.

PLACEHOLDER — REPLACE WITH ACTUAL SCREENSHOT

What to capture:

- Open pgAdmin at `http://localhost:5050` (login: see `docker-compose.yml`)
- Navigate to: **Servers** → **techpulse_db** → **Schemas** → **public** → **Tables** → **documents**
- Right-click `documents` → **View/Edit Data** → **All Rows**
- Run ingestion first if table is empty: `docker exec -it scheduler python ingest.py`
- The table grid must show **rows from all 5 sources** visible in the `source` column: `arxiv`, `hn`, `devto`, `github`, `rss`
- Visible columns must include: `id`, `source`, `title`, `state`, `published_at`
- Scroll or resize so the `source` column is clearly readable
- Capture both the pgAdmin left-panel tree (showing database name) and the data grid

Quick query to verify all 5 sources present:

```
SELECT source, COUNT(*) FROM documents GROUP BY source ORDER BY source;
```

Save as: `report/figures/demo_pgadmin.png` (PNG, min 1400px wide to show both panels)

Figure 6.6. pgAdmin: the `documents` table populated with records from all five data sources.

This chapter documented implementation status across 20 components, deviations from the original proposal, and key engineering observations. All core pipeline components are integration-tested; cloud deployment is in progress. Chapter 7 presents remaining work and the project timeline.

CHAPTER 7

REMAINING WORK AND LIMITATIONS

7.1 Remaining Work

The remaining work is organized into four phases spanning approximately four weeks. Table 7.1 provides a phase-level overview, followed by detailed task descriptions.

Table 7.1. Remaining Work: Phase Overview.

Phase	Focus	Timeline	Key Deliverables
A	System Validation & Corpus Building	Mar W3–W4	Populated corpus; local integration verified
B	Evaluation & Experimentation	Mar W4–Apr W1	Empirical results; grid search output
C	Analysis, Monitoring & Dashboard	Apr W1–W3	Drift validation; results dashboard; visualizations
D	Final Report & Submission	Apr W2–W3	Completed report with empirical evidence

7.1.1 Phase A: System Validation and Corpus Building (Mar W3 – Mar W4)

1. **Corpus population via multi-cycle ingestion.** Execute scheduled ingestion cycles across all five data sources over a sustained period to build a sufficiently large and diverse corpus. Target: 1,000+ indexed documents with balanced source representation across ArXiv, Hacker News, DEV.to, GitHub Trending, and RSS feeds.

2. **Local Docker Compose integration testing.** Verify end-to-end pipeline functionality in the local Docker Compose environment: ingestion → preprocessing → embedding → indexing → retrieval → generation. Confirm all six services (db, pgadmin, localstack, api, frontend, scheduler) operate correctly together.

3. **Frontend integration verification.** Test the React frontend against the live API to verify citation-grounded response rendering, baseline/hybrid mode toggling, abstention message handling, source badge display, and retrieval metadata transparency.

4. **Cloud deployment (in progress).** Deploy the full system to the AWS Free Tier. The deployment uses `template-freetier.yaml` and `samconfig-freetier.toml`. The SAM template has been prepared and validated, and initial stack creation is underway. Remaining: verify Lambda function execution, RDS connectivity, S3 storage operations, SQS message flow, and API Gateway routing under cloud conditions.

7.1.2 *Phase B: Evaluation and Experimentation (Mar W4 – Apr W1)*

5. **Full baseline evaluation run.** Execute 50 evaluation queries in vector-only (baseline) mode against the populated corpus. Record context precision, faithfulness, citation grounding ratio, and per-query latency for each query.

6. **Full hybrid evaluation run.** Execute the same 50 evaluation queries in hybrid mode using the default reranking weights ($\alpha = 0.6$, $\beta = 0.2$, $\gamma = 0.2$). Record all metrics under identical corpus and embedding conditions to enable paired comparison.

7. **Grid search weight optimization.** Run the reranking weight grid search across $\alpha \in \{0.4, 0.5, 0.6, 0.7\}$ with the latency constraint (p95 $\leq$ 2.0s). Identify the optimal weight configuration that maximizes mean context precision.

8. **Sensitivity analysis.** Execute one-at-a-time parameter sweep across α, β, and γ through $[0.0, 1.0]$ in 0.1 steps (33 runs) to quantify the marginal influence of each parameter and justify weight selection beyond grid search.

9. **Statistical testing.** Apply Wilcoxon signed-rank test, Cohen's d effect size, and bootstrap 95% confidence intervals to quantify significance and practical impact of baseline-vs-hybrid differences.

10. **Latency profiling and cost projection.** Decompose end-to-end latency by stage (metadata filtering, vector search, reranking, LLM generation). Project monthly costs for 50/100/200 queries/day scenarios against free-tier ceilings.

7.1.3 *Phase C: Analysis and Monitoring Validation (Apr W1 – Apr W2)*

11. **Drift detection validation.** Execute multiple drift detection cycles against the populated corpus to validate the dual-criteria mechanism (10% relative threshold + Shewhart 3σ). Confirm that alerts trigger correctly under simulated drift conditions and that baseline history accumulates as expected.

12. **Results analysis and visualization.** Produce comparison tables and visualizations: baseline vs. hybrid precision paired differences, grid search Pareto frontier (precision vs. latency), per-source retrieval distribution, and latency breakdown charts.

13. **System demonstration screenshots.** Capture screenshots for all placeholder figures in the report (Figures 6.1–6.6 and Appendix B), replacing placeholder boxes with actual system output from Docker Compose and the AWS Free Tier environment.

7.1.4 *Phase D: Final Report and Submission (Apr W2)*

14. **Evaluation chapter completion.** Write the complete evaluation results chapter with empirical findings, statistical comparison of baseline vs. hybrid retrieval, discussion of grid search outcomes, and analysis against the four evaluation targets (precision > 0.75, high faithfulness, p95 < 2s, cost $<$ USD 15/month).

15. **Discussion and conclusions.** Complete the discussion of results including the five planned analyses (Section 5.5): precision vs. overhead, retrieval strictness vs. recall, cost–performance balance, weight sensitivity, and production suitability assessment.

16. **Final report revision and submission.** Integrate all empirical content, screenshots, and analysis into the final report. Conduct internal peer review, address formatting and consistency, and prepare the submission package including source code archive.

7.2 Known Limitations

1. The evaluation corpus is limited to five technology-focused data sources; generalizability to other domains or larger corpora requires further study.

2. The evaluation query set of 50 curated queries across three balanced categories (research_trend, tool_technology, comparative) provides sufficient statistical power for the paired Wilcoxon test. However, a larger and more diverse query set would further strengthen external validity.

3. Amazon Bedrock is available on the Free Tier but has not yet been evaluated as the primary backend. The primary backend (Groq Llama 3.1 8B Instant) is used for all evaluations. Since the prompt template, evaluation methodology, and RAGAS metrics are backend-agnostic, the comparative findings between baseline and hybrid retrieval remain valid and transferable across backends.

4. The `db.t3.micro` RDS instance provides limited compute and memory compared to Aurora Serverless. Under high ingestion volumes, database performance may become a bottleneck.

5. Cross-encoder reranking and ANN-optimized indexing strategies are out of scope for this project but identified as future research directions.

6. Budget governance via Cost Explorer has higher latency than the originally proposed AWS Budgets integration, as Cost Explorer data may lag by up to 24 hours.

7. The current unit test suite (200 tests across 15 modules) covers core logic but lacks integration tests that exercise the full pipeline end-to-end with a live database and AWS services.

8. **Context window management.** The current implementation retrieves top-$k = 5$ chunks at 500 tokens each, consuming approximately 2,600 prompt tokens. While this fits comfortably in Groq's 128k-token context window and Bedrock Nova Micro's 128k-token window, for Ollama-hosted Llama 3.2 3B (configured with 4,096-token context window), context may be truncated silently. A token budget check before prompt construction is identified as a near-term improvement.

9. **Secrets management.** API keys (Groq token, HuggingFace token, GitHub token) are currently stored as environment variables in Lambda configurations. For production hardening, migration to AWS Secrets Manager or SSM Parameter Store is recommended to avoid key exposure in infrastructure-as-code templates.

7.3 Project Timeline

Figure 7.1 presents the project timeline, including completed milestones and remaining work.

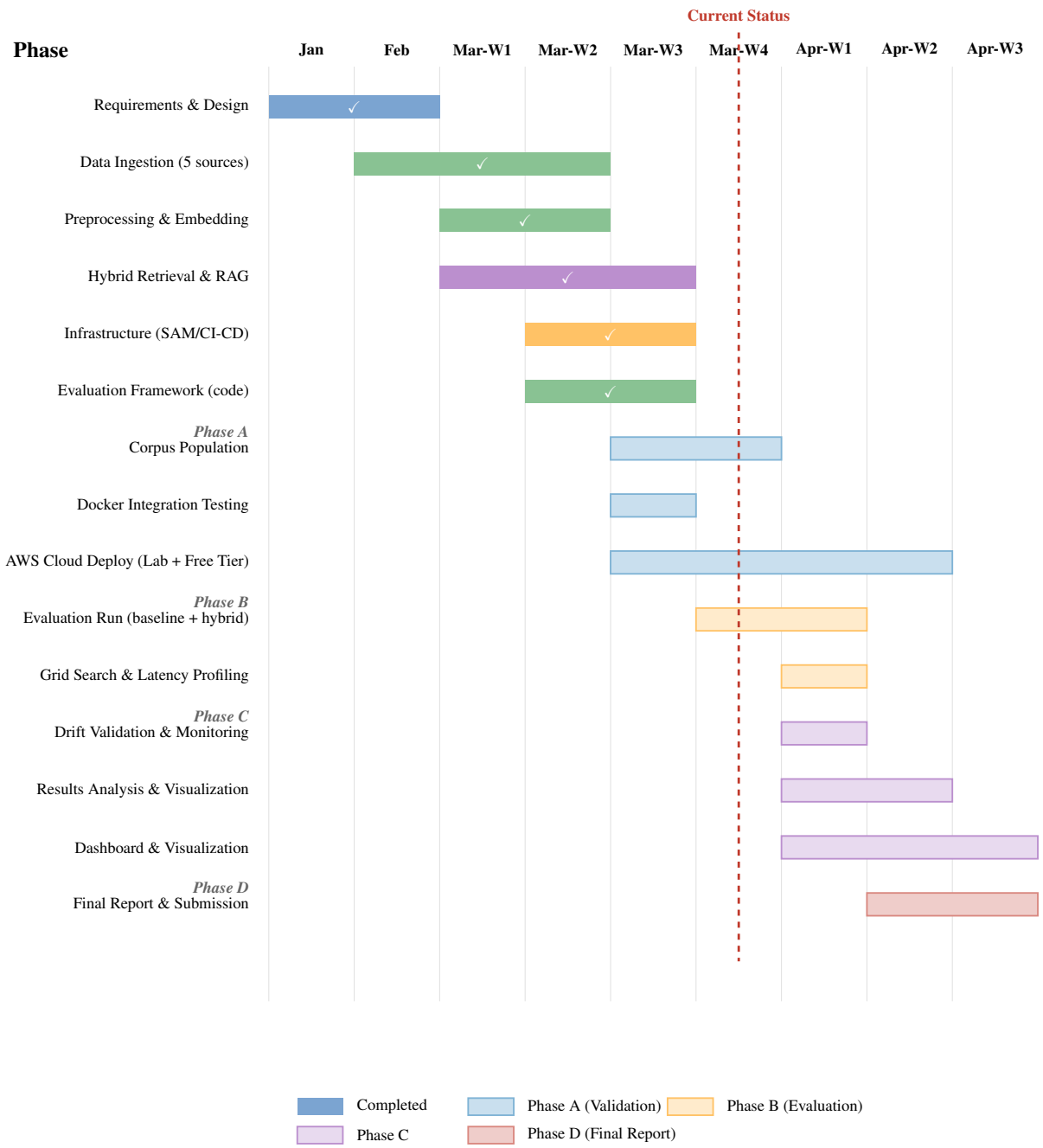

Figure 7.1. Project timeline (Gantt chart). Solid bars indicate completed phases; outlined bars indicate remaining work (Phases A–D).

7.4 Future Extensions

Several extensions are identified for post-prototype development:

- Migration to Aurora Serverless (applicable when scaling beyond Free Tier constraints)

- Full evaluation with Amazon Bedrock (Nova Micro) as the primary backend via the Free Tier deployment
- Expansion of data sources (StackExchange, academic conference proceedings)
- Cross-encoder reranking evaluation for precision-critical use cases
- Embedding model fine-tuning or replacement based on drift analysis findings
- Integration testing suite with containerized database and mocked AWS services
- User feedback loop for continuous retrieval quality improvement
- Token budget management: pre-prompt token count check to prevent context truncation in constrained-context-window backends
- Secrets Manager / SSM Parameter Store migration for production-grade API key handling

This chapter presented the four-phase completion plan, nine known limitations, and future extension directions. The primary remaining risks—cloud deployment verification and the full evaluation run—are mitigated by the dual-environment setup and the pre-built evaluation codebase.

REFERENCES

AWS. (2023). *AWS well-architected framework*. Amazon Web Services. https://aws.amazon.com/architecture/well-architected/

Bayram, F., Ahmed, B. S., & Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, *245*, 108632. https://doi.org/10.1016/j.knosys.2022.108632

Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., de Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A., ... Sifre, L. (2022). Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning* (Vol. 162, pp. 2206–2240). PMLR.

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the IEEE International Conference on Big Data* (pp. 1123–1132). IEEE.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.

deepset. (2023). *Haystack: Open-source LLM framework to build production-ready applications*. deepset GmbH. https://haystack.deepset.ai/

Es, S., James, J., Anke, L. E., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, *46*(4), 1–37. https://doi.org/10.1145/2523813

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2022). Atlas: Few-shot learning with retrieval augmented language models. *Advances in Neural Information Processing Systems*, *35*, 22251–22265.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38. https://doi.org/10.1145/3571730

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Liu, J. (2022). *LlamaIndex: A data framework for LLM applications*. `https://www.llamaindex.ai/`

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, *31*(12), 2346–2363. `https://doi.org/10.1109/TKDE.2018.2876857`

Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9004–9017). Association for Computational Linguistics.

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-T., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12076–12100). Association for Computational Linguistics.

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2023). ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, *28*, 2503–2511.

Shao, Y., Jiang, Y., Kanell, T. A., Xu, P., Khattab, O., & Lam, M. S. (2024). Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 6252–6278). Association for Computational Linguistics.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 31210–31227). PMLR.

Thakur, N., Reimers, N., Daxenberger, J., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the 35th Conference on Neural Information Processing Systems—Datasets and Benchmarks Track*.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, *33*, 5776–5788.

Weaviate. (2023). *Verba: The golden RAGtriever*. Weaviate BV.

https://github.com/weaviate/Verba

Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, *11*, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138

LangChain. (2023). *LangChain: Build context-aware reasoning applications*. https://www.langchain.com/

Zhang, Y., Chen, X., Zhang, Y., Hu, X., Liu, Z., & Tang, J. (2024). A comprehensive survey on retrieval-augmented generation. *ACM Transactions on Information Systems*. Advance online publication. https://doi.org/10.1145/3703155

APPENDIX A

SOURCE CODE AVAILABILITY

The implementation of *TechPulse: Hybrid Retrieval-Augmented Generation System for Emerging Technology Intelligence* has been developed using Python 3.13 and modern cloud-native technologies.

To support reproducibility, transparency, and collaborative development, the project source code is maintained in a GitHub organization repository:

`https://github.com/EZ2-555555/data-pipeline-free-tier`

The repository includes the following implemented components:

- Five data ingestion modules for collecting data from ArXiv API, Hacker News API, DEV.to API, GitHub Trending API, and curated RSS/Atom feeds
- A 17-step preprocessing pipeline for comprehensive text normalization and token-based chunking using tiktoken (`cl100k_base`)
- Semantic embedding pipeline using the `all-MiniLM-L6-v2` sentence-transformer model (384-dimensional embeddings)
- Hybrid retrieval implementation including metadata filtering, cosine similarity search via pgvector, and reranking-lite scoring with configurable weights
- RAG orchestration with three-layer hallucination verification (prompt grounding, RAGAS faithfulness, citation traceability)
- Automatic LLM backend fallback chain supporting Groq, Bedrock, Ollama, and HuggingFace Inference API
- Dual-criteria drift detection combining Shewhart 3σ control charts with relative-drop thresholds
- Infrastructure-as-Code definitions using AWS SAM for Free Tier deployment (15+ AWS resources)
- React-based frontend interface for query submission and citation-grounded response display
- Evaluation framework with 50 curated queries (3 categories), 9-phase evaluation pipeline including RAGAS metrics, grid search, sensitivity analysis, statistical tests, drift validation, composite metric, and cost projection
- Four-stage CI/CD pipeline using GitHub Actions (lint, test, build, deploy)
- 200 unit tests across 15 test modules with full pass status
- Docker Compose configuration for local development with 6 services

APPENDIX B

SYSTEM DEMONSTRATION SCREENSHOTS

This appendix provides supplementary screenshots demonstrating additional system functionality.

Health Check Endpoint

Figure B.1 shows the `/health` API endpoint response, confirming connectivity to all four service dependencies (PostgreSQL, S3, SQS, and LLM backend).

Screenshot: Browser or curl output of `GET /health` showing DB, S3, SQS, and LLM status.

File to place: `figures/demo_health.png`

Figure B.1. Health check endpoint (`/health`): JSON response confirming database connectivity, document/chunk counts, S3 reachability, SQS reachability, and LLM backend status.

Drift Detection Endpoint

Figure B.2 shows the `/drift` API endpoint returning a drift detection report, demonstrating the dual-criteria Shewhart 3σ monitoring mechanism.

Screenshot: Browser or curl output of `GET /drift` showing mean similarity, alert status, and probe results.

File to place: `figures/demo_drift.png`

Figure B.2. Drift detection endpoint (`/drift`): JSON response showing probe query mean similarity, standard deviation, alert status, and Shewhart 3σ control analysis.

API: RAG Query Response

Figure B.3 shows the raw JSON response from the `/ask` endpoint, displaying the structured output including the generated answer, source metadata, similarity scores, and retrieval mode.

Screenshot: Browser or curl output of `POST /ask` showing full JSON response with answer, sources, and scores.

File to place: `figures/demo_api.png`

Figure B.3. RAG query endpoint (`POST /ask`): full JSON response showing the generated answer, retrieved source documents with titles, URLs, similarity scores, and retrieval mode metadata.